PES UNIVERSITY ONLINE

Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 3

Preet Kanwal

Department of Computer Science & Engineering

Example 1:

Find out whether the given grammar is ambiguous or not?

1. S⟶ aS|Sa|λ
2. S⟶ aSbS|bSaS|λ
3. R⟶ R+R|RR|R*|a|b|c
4. S⟶ 1S|11S|λ
5. S⟶ AB|aaB
   A⟶ a|Aa
   B⟶ b

Solution :

1. aa
2. abab
3. a+bc
4. 111
5. aab

Example 2:
Show that union of $\{a^n b^n c^m | n>=0, m>=1\}$ $\{a^n b^m c^m | n>=1, m>=0\}$ is inherently ambiguous.

Solution :

$L=\{a^n b^n c^m\} \cup \{a^n b^m c^m\}$

$S1 \rightarrow Ac$ $\quad$ $S2 \rightarrow aB$

$A \rightarrow aAb|\lambda$ $\quad$ $B \rightarrow bBc|\lambda$

$S \rightarrow S1|S2$

- Strings that belong to both the languages are $L1 \cap L2=\{a^n b^n c^n\}$
- For every string in $a^n b^n c^n$ there exists two parse trees(either using LMD or RMD).
- No other grammar can generate L.Hence ,the language L is ambiguous or we can say that the given language L is inherently ambiguous.

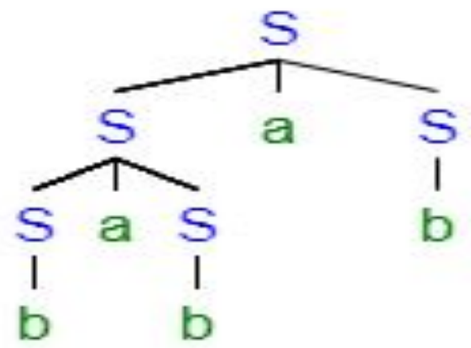
Example 3:

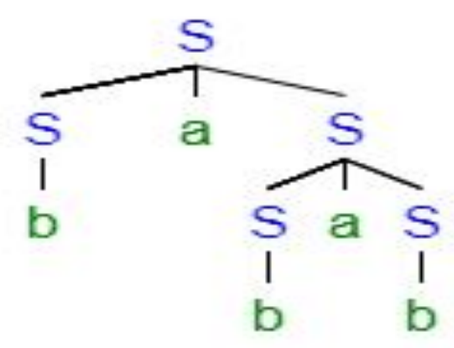
Find out whether the given grammar is inherently ambiguous or not. S→ SaS|b

Solution :

➔ **First we prove the grammar is ambiguous by showing that a string has more than one parse tree**

Consider the string ”babab”,to prove the grammar is ambiguous.





Two parse trees for the string “babab” indicates the grammar is ambiguous.

Try constructing another grammar which is unambiguous.
The language is regular ,we can write the regular expression b(ab)+
S⟶ bA
A⟶ abA|ab
Now,the grammar is unambiguous.

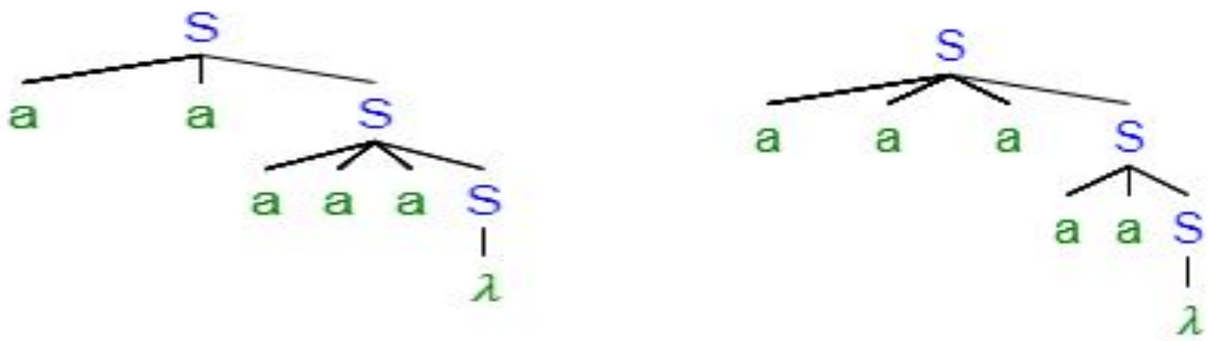
Example 4:

Find out whether the given grammar is inherently ambiguous or not. S→ aaS|aaaS|λ

Solution :

➔ First we prove the grammar is ambiguous by showing that a string has more than one parse tree. .

Parse tree for the string “aaaaa”.



Two parse trees for the string “aaaaa” indicates the grammar is ambiguous.

➔ Try constructing another grammar which is unambiguous.

L=(aa+aaa)*
S⟶ aaA|λ
A⟶ aA|λ

This grammar is unambiguous.

Example 5:

Find out whether the given grammar is inherently ambiguous or not.

S⟶ AB|aaB

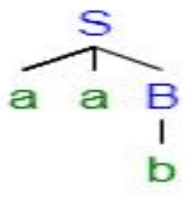
A⟶ a|Aa

B⟶ b

Solution :

First we prove the grammar is ambiguous by showing that a string has more than one parse tree.

Parse tree for the string “aab”



Two parse trees for the string “aab” indicates the grammar is ambiguous.

➔ Try constructing another grammar which is unambiguous

L={ab,aab,aaaaaaaab,.....}

L=aa*b

S⟶ aAb

A⟶ aA|λ

This grammar is unambiguous.

Example 6:
Eliminate ambiguity in following grammar:
B⟶ B or B | B and B| not B | True |False

Solution :

Unambiguous grammar ,

B⟶ B or F | F

F⟶ F and G | G

G⟶ not G|True | False

Example 7:
Eliminate ambiguity in following grammar:
R⟶ R+R|RR|R*|a|b|c

Solution :

Unambiguous grammar,

R⟶ R+S|S

S⟶ S.T|U

U⟶ U*|a|b|c

THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724

PES UNIVERSITY ONLINE

Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 3

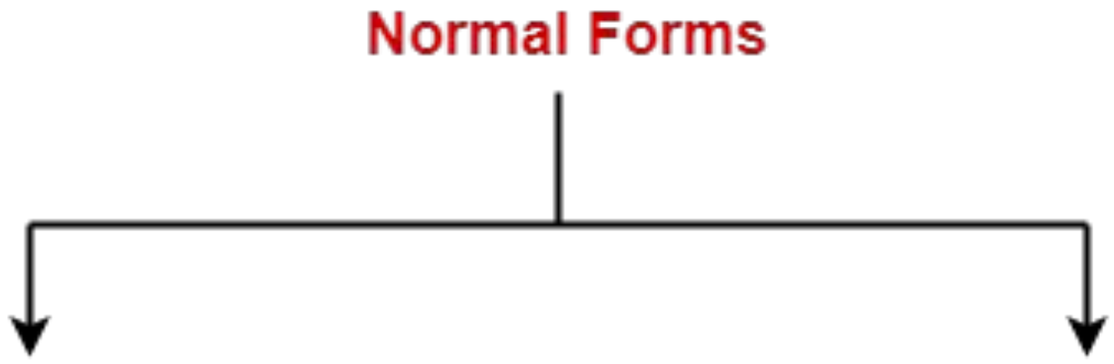
Preet Kanwal

Department of Computer Science & Engineering

Normalization is performed in order to standardize the grammar.

This is because the RHS of productions have no specific format (parsing is hard!).

Idea is to make every step in derivation useful.



https://www.gatevidyalay.com/chomsky-normal-form-normal-forms/#:~:text=This%20is%20because%20the%20RHS,is%20performed%20using%20normal%20forms.

There are at least two relevant uses.

1. **Simplicity of proofs**

   There are plenty of proofs around context-free grammars. Normal forms can be helpful there. It allows determination of:

   - **Membership problem –Is string w is a member of language L(G)?**
   - **Emptiness problem –Is L(G) = ∅?**
   - **Finiteness problem –Is language L(G) finite?**
   - **CNF helps prove that a language is not context-free.**

https://cs.stackexchange.com/questions/10468/the-importance-of-normal-forms-like-chomsky-normal-form-for-cfgs

There are at least two relevant uses.

2. Enables parsing

Normal forms can give us more structure to work with, resulting in easier parsing algorithms.

- CNF is used by efficient parsing algorithm (called CYK algorithm)
- In a GNF grammar, any derivable string of length n can be produced with exactly n production steps.

https://cs.stackexchange.com/questions/10468/the-importance-of-normal-forms-like-chomsky-normal-form-for-cfgs

- CNF restricts the number of symbols on the right side of a production to be two.
- It makes the parse tree for derivations using this form of the CFG a binary tree.
- The key advantage is that in Chomsky Normal Form, every derivation of a string of n letters has exactly $2n - 1$ steps.
- Thus, one can determine if a string is in the language by exhaustive search of all derivations.

- **For a given grammar, there can be more than one CNF.**
- **CNF produces the same language as generated by CFG.**
- **Every grammar in Chomsky normal form is context-free, and conversely, every context-free grammar can be transformed into an equivalent one which is in Chomsky normal form and has a size no larger than the square of the original grammar's size.**

A context free grammar is in chomsky normal form (CNF) if all the productions rules satisfy one of the following condition

1) **A non - terminal generating terminal**
   Ex: X->x
2) **A non - terminal generating two non terminals**
   Ex: X-> XY
3) **Only Start symbol can generate λ, if λ is a part of the language**
   Ex: S->λ

Automata Formal Languages and Logic

Unit 3 - Steps to convert CFG to Chomsky Normal Form

Step 1: Eliminate lambda productions

(Every step must be useful)

Step2: Eliminate Unit production

(Each step must either increase the length of the sentential form or the number of terminals)

Step 3: Eliminating Useless productions and Symbols

(

There are 2 aspects :

1. **Derivability : Each Variable must derive a string/must end with set of terminals or lambda.**
2. **Reachability : Each Variable must be reachable from S.**

)

Remember : Lambda Units are Useless !!

Step 4: Convert the CFG to CNF as per definition.

Example 1:

S -> aX | Yb
X ->S | λ
Y -> bY | b

Solution :

Step 1:

Eliminate λ productions

S -> aX | a | Yb
X-> S
Y-> bY | b

Example 1:

S -> aX | Yb
X ->S | λ
Y -> bY | b

Solution :

Step 2: Eliminate Unit Productions

S -> aX | a | Yb
X -> aX | a | Yb
Y -> bY | b

Example 1:

S -> aX | Yb

X ->S | λ

Y -> bY | b

Solution :

Step 3: There are no useless productions

Step 4: Conversion to CNF:

S-> AX | YB | a

X -> AX | YB | a

Y -> BY | b

A -> a

B -> b

Example 2:

$S \rightarrow aSa \mid bSb \mid A \mid \lambda$

$\mathbf{A} \rightarrow a \mid b \mid \lambda$

Solution :

Step 1: Remove λ production

$S \rightarrow aSa \mid aa \mid bSb \mid bb \mid A$

$\mathbf{A} \rightarrow a \mid b$

Example 2:

$S \rightarrow aSa \mid bSb \mid A \mid \lambda$

$A \rightarrow a \mid b \mid \lambda$

Solution :

Step 2: Remove unit production ($S \rightarrow A$)

$S \rightarrow aSa \mid aa \mid bSb \mid bb \mid a \mid b$

$A \rightarrow a \mid b$

Example 2:

$S \rightarrow aSa \mid bSb \mid A \mid \lambda$

$A \rightarrow a \mid b \mid \lambda$

Solution :

Step 3: Remove useless production(A)

$S \rightarrow aSa \mid aa \mid bSb \mid bb \mid a \mid b$

Now the CFG is

$A \rightarrow a$

$B \rightarrow b$

$S \rightarrow ASA \mid BSB \mid AA \mid BB \mid a \mid b$

Example 2:

$S \rightarrow aSa \mid bSb \mid A \mid \lambda$

$A \rightarrow a \mid b \mid \lambda$

Solution :

Step 4: To CNF

$A \rightarrow a$

$B \rightarrow b$

$C \rightarrow AS$

$D \rightarrow BS$

$S \rightarrow CA \mid DB \mid AA \mid Bb \mid a \mid b$

Example 3:

$S \rightarrow BAB$

$B \rightarrow bba$

$A \rightarrow Bc$

Solution :

Step 1: There are no λ production

Step 2 There are no unit productions

Step 3: There are no useless production

Example 3:

$S \rightarrow BAB$

$B \rightarrow bba$

$A \rightarrow Bc$

Solution :

Step 4: To CNF

$C \rightarrow a$

$D \rightarrow b$

$E \rightarrow c$

$F \rightarrow BA$

$G \rightarrow DD$

$S \rightarrow FB$

$B \rightarrow GC$

$A \rightarrow BE$

Example 4:

$S \rightarrow Aa \mid B \mid Ca$

$B \rightarrow aB \mid b$

$C \rightarrow Db \mid D$

$D \rightarrow E \mid d$

$E \rightarrow ab$

Solution :

Step 1: There are no λ production

Step 2: Remove unit production

$S \rightarrow Aa \mid aB \mid b \mid Ca$

$B \rightarrow aB \mid b$

$C \rightarrow Db \mid ab \mid d$

$D \rightarrow ab \mid d$

$E \rightarrow ab$

Example 4:

S →Aa | B | Ca

B → aB | b

C → Db | D

D → E | d

E → ab

Solution :

Step 3: Remove useless production

E is useless production and Aa is useless as there is no variable A

Step 4: To CNF

X → a

Y → b

S → XB | b | CX

B → XB | b

C → DY | XY | d

D → XY | d

Example 5:

$S \rightarrow aAa \mid bBb \mid BB$

$A \rightarrow C$

$B \rightarrow S \mid A$

$C \rightarrow S \mid \lambda$

Solution :

Step 1: Remove λ production

$S \rightarrow aAa \mid aa \mid bBb \mid bb \mid BB$

$A \rightarrow C$

$B \rightarrow S \mid A$

$C \rightarrow S$

Example 5:

$S \rightarrow aAa \mid bBb \mid BB$

$A \rightarrow C$

$B \rightarrow S \mid A$

$C \rightarrow S \mid \lambda$

Solution :

step 2: Remove unit productions

$S \rightarrow aAa \mid aa \mid bBb \mid bb \mid BB$

$A \rightarrow aAa \mid aa \mid bBb \mid bb \mid BB$

$B \rightarrow aAa \mid aa \mid bBb \mid bb \mid BB$

$C \rightarrow aAa \mid aa \mid bBb \mid bb \mid BB$

Example 5:

$S \rightarrow aAa \mid bBb \mid BB$

$A \rightarrow C$

$B \rightarrow S \mid A$

$C \rightarrow S \mid \lambda$

Solution :

Step 3: Remove useless production

C is useless production

$S \rightarrow aAa \mid aa \mid bBb \mid bb \mid BB$

$A \rightarrow aAa \mid aa \mid bBb \mid bb \mid BB$

$B \rightarrow aAa \mid aa \mid bBb \mid bb \mid BB$

Example 5:

$S \rightarrow aAa \mid bBb \mid BB$

$A \rightarrow C$

$B \rightarrow S \mid A$

$C \rightarrow S \mid \lambda$

Solution :

Step 4: To CNF

$X \rightarrow a$

$Y \rightarrow b$

$P \rightarrow XA$

$Q \rightarrow YB$

$S \rightarrow PX \mid XX \mid QY \mid YY \mid BB$

$A \rightarrow PX \mid XX \mid QY \mid YY \mid BB$

$B \rightarrow PX \mid XX \mid QY \mid YY \mid BB$

Example 6:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow num \mid id$

Solution :

Step 1: There are no λ production

Example 6:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow num \mid id$

Solution :

Step 2: Remove unit production ($E \rightarrow T$) and($T \rightarrow F$)

$E \rightarrow E + T \mid T * F \mid F$

$T \rightarrow T * F \mid num \mid id$

$F \rightarrow num \mid id$

This results in $E \rightarrow F$, remove this

$E \rightarrow E + T \mid T * F \mid num \mid id$

$T \rightarrow T * F \mid num \mid id$

$F \rightarrow num \mid id$

Example 6:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow num \mid id$

Solution :

Step 3: There are no use less productions

Now the CFG is:

$A \rightarrow +$

$B \rightarrow *$

$E \rightarrow EAT \mid TBF \mid num \mid id$

$T \rightarrow TBT \mid num \mid id$

$F \rightarrow num \mid id$

Example 6:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow num \mid id$

Solution :

Step 4 : To CFG

$A \rightarrow +$

$B \rightarrow *$

$C \rightarrow EA$

$D \rightarrow TB$

$E \rightarrow CT \mid DF \mid num \mid id$

$T \rightarrow DT \mid num \mid id$

$F \rightarrow num \mid id$

THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724

PES UNIVERSITY ONLINE

Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 3

Preet Kanwal

Department of Computer Science & Engineering

Example 1:

Parse the string abba using CYK algorithm ,

Grammar:

S -> aSb | bSa | SS | λ

Solution :

Conversion to CNF

S -> AB | BA | AC | BD | SS | λ

A -> a

B -> b

C -> SB

D -> SA

Example 1:

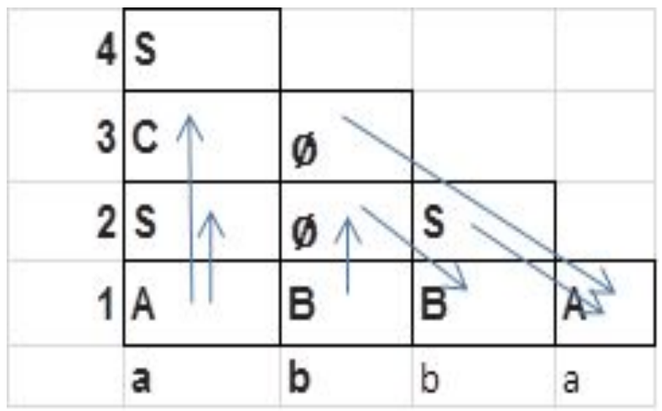
Parse the string abba using CYK algorithm

,

Grammar:

S -> aSb | bSa | SS | λ

Solution :



1) **Strings of the length 1 can be generated by**
   A -> a
   B -> b
2) **Strings of the length 2 can be generated by**
   For AB
   S -> AB
   For BA
   S -> BA
   For BB it is ⌀
3) **Strings of the length 3 can be generated by**
   a) **A . ⌀ ∪ S . B = ⌀ . SB (SB is generated by C)**
      C -> SB
   b) B .S ∪ ⌀ . A
      BS is not generated by any rule
4) **Strings of the length 4 can be generated by**

A . ⌀ ∪ SS ∪ CA

S -> SS

The given string belongs to the grammar

Example 1:

Parse the string aabba using CYK algorithm ,

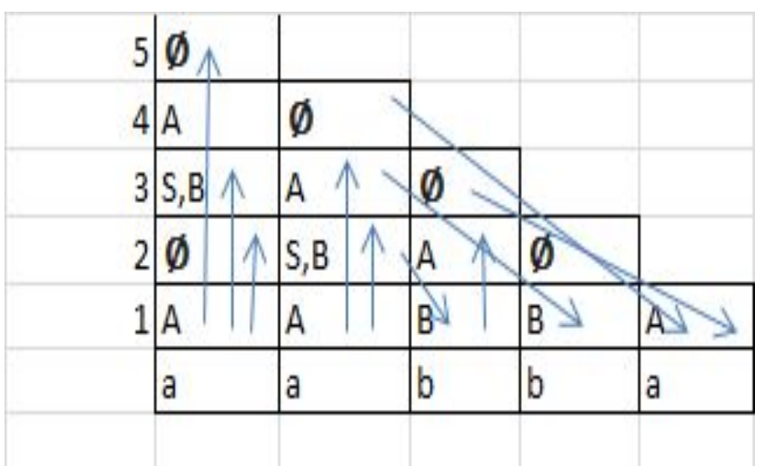
Grammar:

S -> AB

A -> BB | a

B -> AB | b

Solution :



Length 3:

1) A (S, B) ∪ ⌀**.B** **(S -> AB, B -> AB)**
   AS, AB , ⌀
2) **AA** ∪ (S ,B) (B) (A -> BB)
   AA ∪ SB,BB
3) A . ⌀
   ⌀

Length 4:

1) **AA** ∪ ⌀**. A** ∪ (S,B) (B)
   AA ∪ ⌀ ∪ SB ∪ BB
   (A -> BB)
2) A. ⌀ ∪ (S,B) ⌀ ∪ AA
   ⌀ ∪ ⌀ ∪ AA
   =⌀

Length 5:

A ⌀ ∪ ⌀ . ⌀ ∪ (S,B) ⌀ ∪ AA

= ⌀

The string does not belong to grammar

Example 1:

Parse the string baaa using CYK algorithm ,

Grammar:

S → AA | BC

A → BA | a

B→ CC | b

C → AB | a

Solution :

4	S,A,C			
3	Ø	S,A,C		
2	A,S	B	B	
1	B	A,C	A,C	A,C
	b	a	a	a

Length 3:

1) BB ∪ {A, S }{A,C}
   BB ∪ AA ∪ AC ∪ SA ∪ SC
   =⌀
2) **(A,C) (B)** ∪ B(A,C)
   AB ∪ CB ∪ BA ∪BC
   S → AB | BC A → BA
   C → AB

Length 4:

B(S,A,C) ∪ (A,S) B ∪ ⌀ **(A,C)**
= BS ∪ BA ∪ BC ∪ AB,SB
A → BA
S → BC
S → AB
C → AB

The string baaa belongs to the grammar

THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724

PES UNIVERSITY ONLINE

Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 3

Preet Kanwal

Department of Computer Science & Engineering

A context-free grammar is in **Greibach normal form** (GNF) if the right-hand sides of all production rules start with a terminal symbol, optionally followed by some variables.

All productions have form:

$$A \rightarrow a\,V_1 V_2 \cdots V_k \qquad k \geq 0$$

symbol variables

https://en.wikipedia.org/wiki/Greibach_normal_form
https://www.cs.colostate.edu/~massey/Teaching/cs301/RestrictedAccess/Slides/301lecture10.pdf

PES UNIVERSITY ONLINE

Examples:

$$S \rightarrow cAB$$

$$A \rightarrow aA \mid bB \mid b$$

$$B \rightarrow b$$

Greibach Normal Form

$$S \rightarrow abSb$$

$$S \rightarrow aa$$

Not Greibach Normal Form

https://www.cs.colostate.edu/~massey/Teaching/cs301/RestrictedAccess/Slides/301lecture10.pdf

Steps to convert a CFG to Greibach normal form (GNF) :

- Eliminate lambda, unit and useless productions (in the sequence mentioned).
- We attempt to show the conversion using basic examples in a brief manner (direct conversion).
- Conversion is a little tedious if we go by the algorithm (**skipped**)

Note : Conversion of a CFG to Greibach normal form (GNF) :

If **G = (V, T, P, S)** is a CFG then,
we can construct another CFG **$G_1 = (V_1, T, P_1, S)$ in GNF** such that,

$$L(G_1) = L(G) - \{\lambda\}$$

For every context-free grammar G where **$\lambda \notin L(G)$**, there exists an equivalent grammar in GNF.

However, a non-strict forms makes an exception to this format restriction for allowing the empty word (λ) to be a member of the described language ($S \rightarrow \lambda$).

http://www.iitg.ac.in/gkd/ma513/oct/oct18/note.pdf
https://en.wikipedia.org/wiki/Greibach_normal_form

PES UNIVERSITY ONLINE

Conversion to Greibach Normal Form:

$$S \rightarrow abSb$$

$$S \rightarrow aa$$

Not Greibach Normal Form

$$S \rightarrow aBSB$$
$$B \rightarrow b$$
$$S \rightarrow aA$$
$$A \rightarrow a$$

Greibach Normal Form

https://www.cs.colostate.edu/~massey/Teaching/cs301/RestrictedAccess/Slides/301lecture10.pdf

Example 1:

G : S → aSa | bSb | SS | λ

Solution :

We get $L(G_1) = L(G) - \{\lambda\}$ in a stricter Form

The grammar G_1 in GNF is

1) **Eliminate Lambda Production:**

G_1 : S → aSa | bSb | S | SS | aa | bb

2) **Eliminate Unit Production (S → S)**

G_1 : S → aSa | bSb | SS | aa | bb

3) **No useless production**

Example 1 (continued):

G : S → aSa | bSb | SS | λ

Solution :

We get $L(G_1) = L(G) - \{\lambda\}$ in a stricter Form

4) Convert to GNF

Step 1 (Ensure RHS of each productions starts with a Terminal):

S → aSa | bSb | aSaS | bSbS | aaS | bbS | aa | bb λ

Step 2 (conversion to GNF):

S → aSA | bSB | aSAS | bSBS | aAS | bBS | aA | bB | λ

A → a

B → b

Example 2:

$S \rightarrow XY \mid Xn \mid p$

$X \rightarrow mX \mid m$

$Y \rightarrow Wn \mid o$

Solution :

- There are no Lambda and Unit productions.
- Variable Y is useless as it doesn't derive a terminal, hence the grammar becomes :

$S \rightarrow Xn \mid p$

$X \rightarrow mX \mid m$

Example 2:

$S \rightarrow XY \mid Xn \mid p$

$X \rightarrow mX \mid m$

$Y \rightarrow Wn \mid o$

Solution :

The grammar in GNF is

Step 1:(Ensure RHS of each productions starts with a Terminal)

$S \rightarrow mXn \mid mn \mid p$

$X \rightarrow mX \mid m$

Step 2 :(conversion to GNF):

$S \rightarrow mXN \mid mN \mid p$

$N \rightarrow n$

$X \rightarrow mX \mid m$

Example 3:

S->aA|bB

B->bB|λ

A->aA|λ

Solution :

Eliminate lambda Productions

S->aA|bB | a | b

B->bB|b

A->aA|a

No Unit and Useless Productions

Above grammar is already in GNF.

Automata Formal Languages and Logic

Unit 3 - CFG(in GNF) to PDA conversion

- Convert given CFG to GNF

- From the Start state(let's say q0) without consuming any input symbol, push the Start Symbol to Stack and transit to next state (let's say q1).

$$\delta(q0, \lambda, Z_0) = (q1, SZ_0)$$

- For every production of the form

$$A \rightarrow a\alpha$$

add the transition :

$$\delta(q1, a, A) = (q1, \alpha)$$

- Finally add the following transition :

$$\delta(q1, \lambda, Z_0) = (qf, Z_0)$$

where, qf is the final state.

- PDA constructed using above method will always have only 3 states.
- Stack contents are the Variables in the sentential form of corresponding string derivation by the grammar.

Example 1:

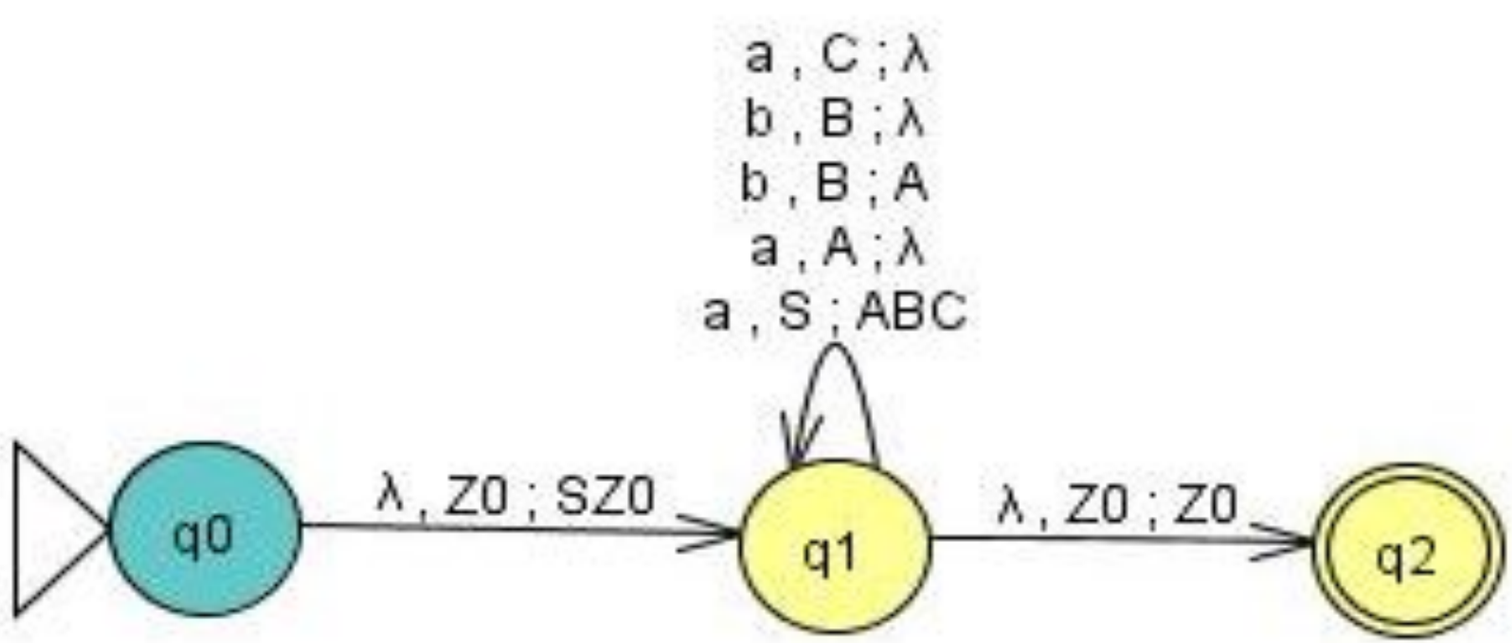
S -> aABC

A -> aB | a

B -> bA | b

C -> a

Solution :



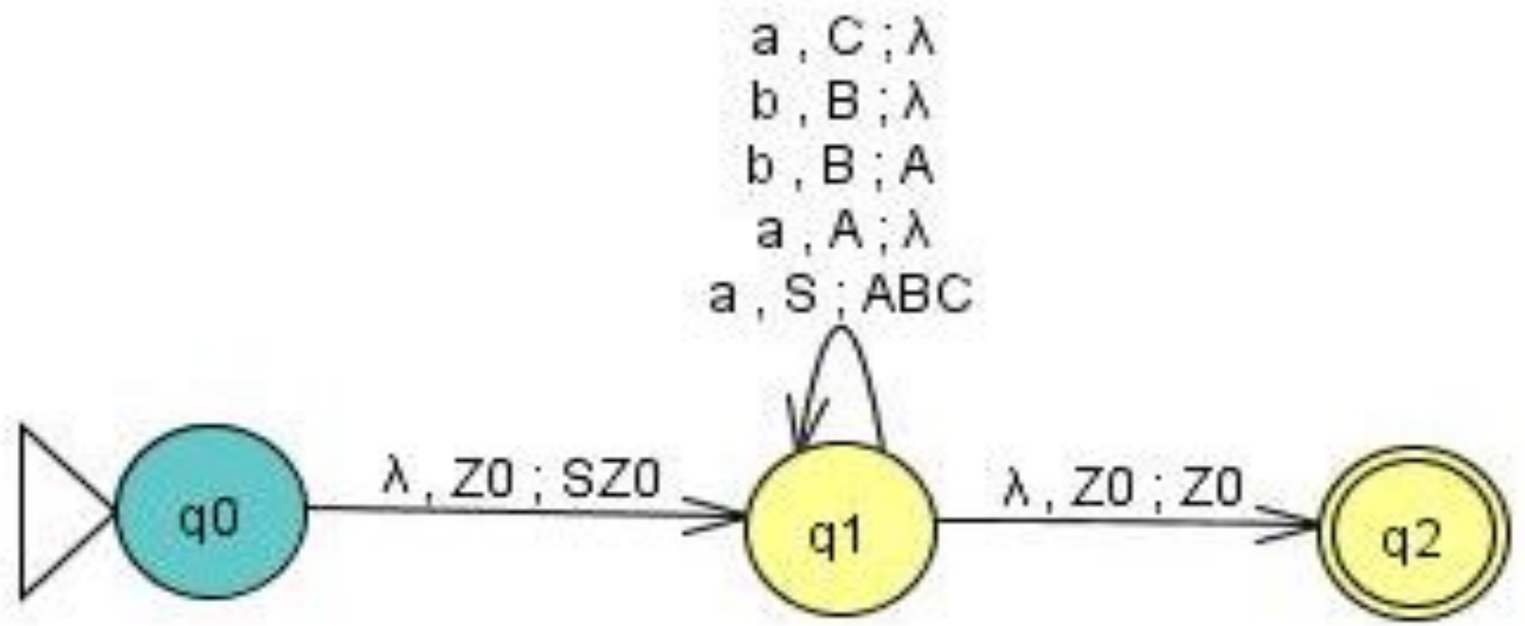
Example 1:

S -> aABC

A -> aB | a

B -> bA | b

C -> a



Trace the string “aababa” on the above PDA.

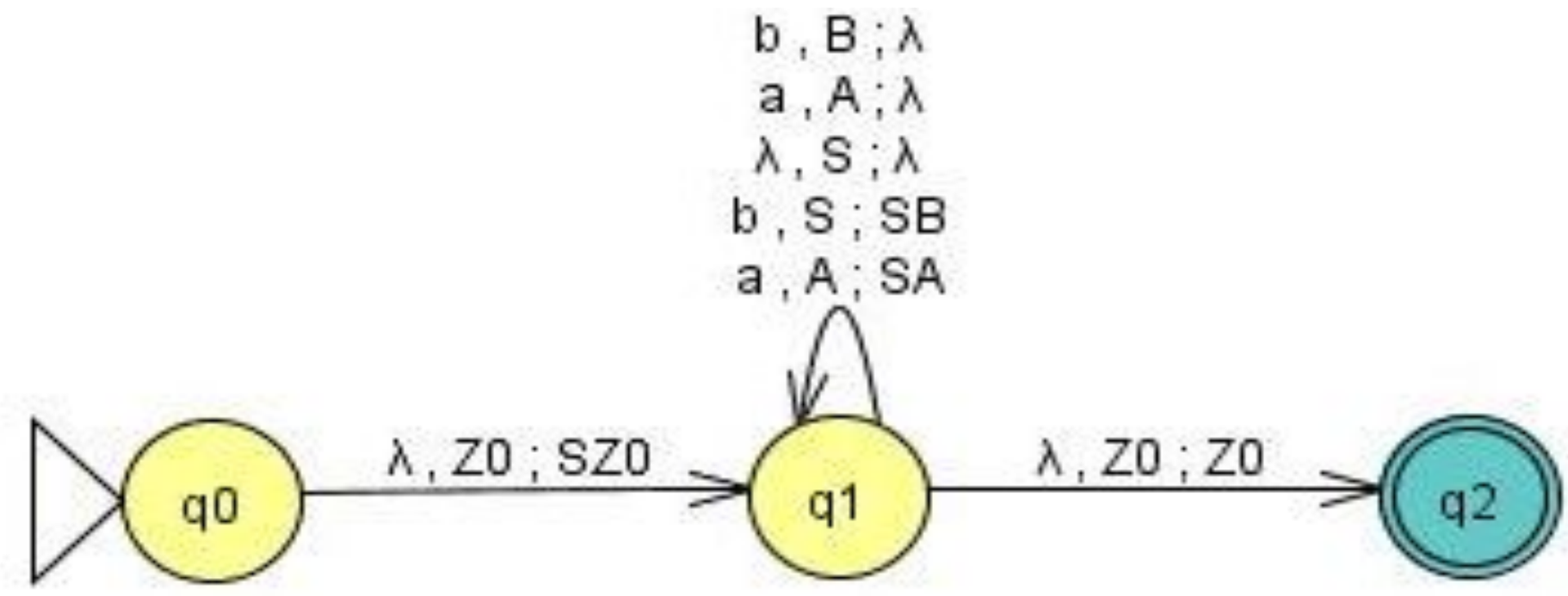
Example 2:

S -> aSA | bSB

A -> a

B -> b

Soluti



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724

PES UNIVERSITY ONLINE

Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 4 - Properties of Context Free Languages

Preet Kanwal

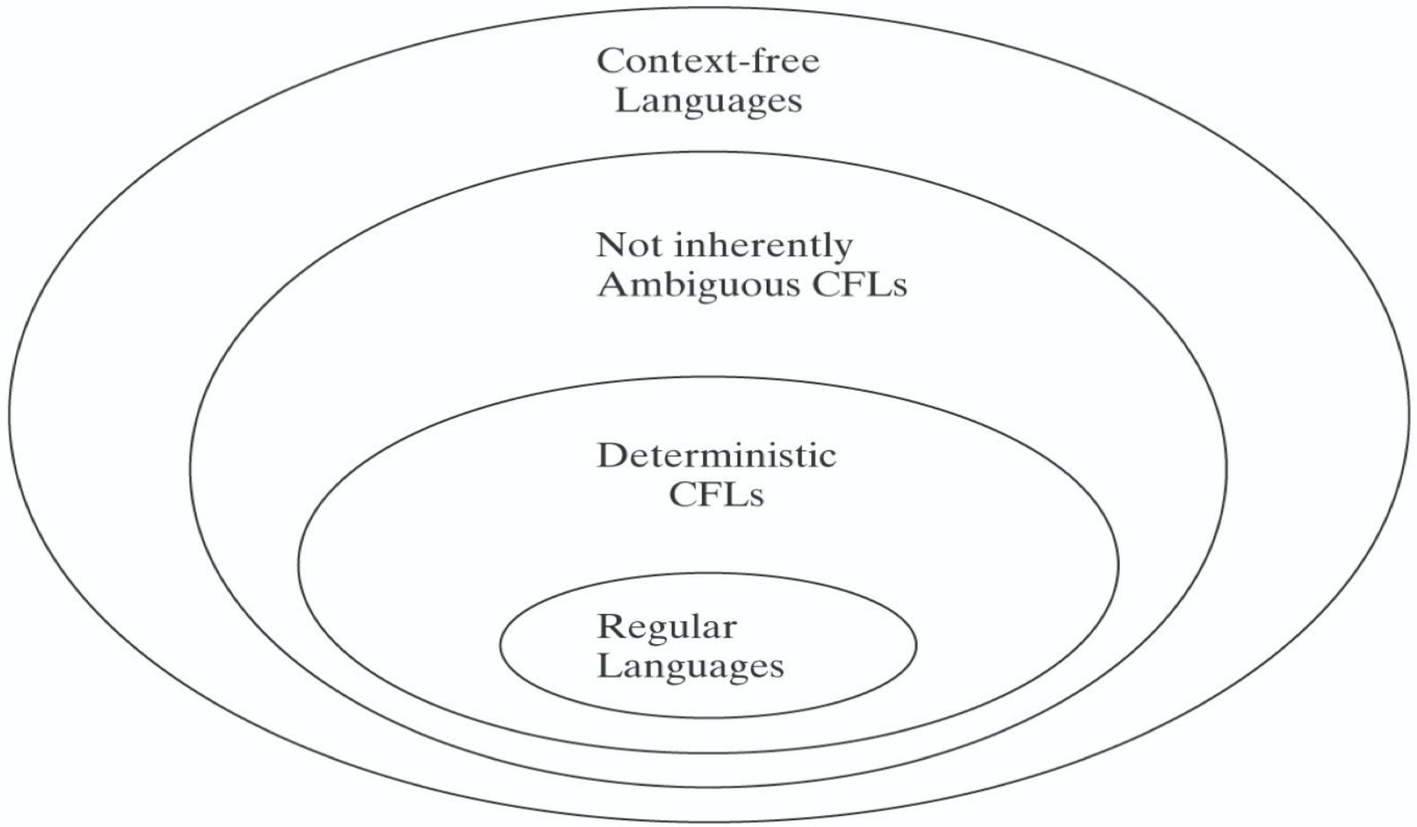
Department of Computer Science & Engineering

- CFLs are of two types:
  1. DCFLs
  2. CFLs*

- CFLs recognize a larger class of grammars.

- DCFLs are proper subsets of CFLs.

- DCFLs are always unambiguous.



*we don't explicitly call these languages as Non-deterministic CFLs(it is understood by default)

Automata Formal Languages and Logic

Unit 4 - Closure properties

Closure properties						
	Union	Concatenation	Star Closure	Reversal	Complement	Intersection
Regular Languages	✓	✓	✓	✓	✓	✓
DCFL	✗	✗	✗	✗	✓	✗
CFL	✓	✓	✓	✓	✗	✗

RL ∩ CFL = CFL

Example:

L_1 = {a*b*} is a Regular Language

L_2 = {$a^n b^n$, n>=0} is a Context Free Language

$L_1 \cap L_2$ = {$a^n b^n$, n>=0} which is a Context Free Language

Assume	**CFG**	$G_1 = (V_1, \Sigma_1 P_1, S_1)$	$G_2 = (V_2, \Sigma_2, P_2, S_2)$
	CFL	$L(G_1) = \{a^n b^n : n \geq 0\}$	$L(G_2) = \{c^n d^n : n \geq 0\}$
	Productions	$S_1 \rightarrow aS_1b$ $S_1 \rightarrow \lambda$	$S_2 \rightarrow cS_2d$ $S_2 \rightarrow \lambda$
Closure under	**Union** $S \rightarrow S_1 \mid S_2$	$G = (V_1 \cup V_2 \cup \{S\}, \Sigma_1 \cup \Sigma_2, P, S)$ where $P = P_1 \cup P_2 \cup \{S \rightarrow S_1 \mid S_2\}$ & S is the new Start symbol. Then, $L(G) = L(G_1) \cup L(G_2) = \{a^n b^n : n \geq 0\} \cup \{c^n d^n : n \geq 0\}$	
	Concatenation $S \rightarrow S_1 S_2$	$G = (V_1 \cup V_2 \cup \{S\}, \Sigma_1 \cup \Sigma_2, P, S)$ where $P = P_1 \cup P_2 \cup \{S \rightarrow S_1 S_2\}$ & S is the new Start symbol. Then, $L(G) = L(G_1)L(G_2) = \{a^m b^m c^n d^n : m,n \geq 0\}$	
	Kleene Star $S \rightarrow S_1 S \mid \lambda$	$G = (V_1 \cup S, \Sigma_1, P, S)$ where $P = P_1 \cup \{S \rightarrow S_1 S \mid \lambda)$ & S is the new Start symbol. Then, $L(G) = L(G_1)^* = \{a^n b^n : n \geq 0\}^*$	
	Reversal	$G = (V_1, \Sigma_1, P_1^R, S_1)$where $P_1^R = S_1 \rightarrow bS_1a \mid \lambda$ (all RHS of the production are reversed) Then, $L(G) = L(G_1)^R = \{b^n a^n : n \geq 0\}^*$	

Unit 4 - CFLs and DCFLs are not closed under Intersection

If L_1 and If L_2 are two context free languages, their intersection L1 ∩ L2 need not be context free. Consider the following CFLs:

$L_1 = \{ a^n b^n c^m : n >= 0, m >= 0 \}$

$L_2 = \{ a^n b^m c^m : n >= 0, m >= 0 \}$

Consider language L which represents the intersection of these two languages :

$L = \{ a^n b^n c^n : n >= 0 \}$

This language L is not a CFL.

L_1 says number of a's should be equal to number of b's and L_2 says number of b's should be equal to number of c's. Their intersection says both conditions need to be true, but push down automata can compare only two. So it cannot be accepted by pushdown automata, hence the language is not context free.

Using DeMorgan's Law:

- $L_1 \cap L_2 = \neg(\neg L_1 \cup \neg L_2)$
- Recall that the context-free languages are closed under union
- So if they were closed under complement, they would also be closed under intersection (which they are not)

Automata Formal Languages and Logic

Unit 4 - DCFLs are not closed under Union

Consider the following DCFLs for which we can easily construct a DPDA:

$L_1 = \{ a^n b^n c^m : n >= 0, m >= 0 \}$

$L_2 = \{ a^n b^m c^m : n >= 0, m >= 0 \}$

Consider language L which represents the union of these two languages :

$L = \{ a^n b^n c^m \cup a^n b^m c^m \mid n >= 0, m >= 0 \}$

This language L cannot be recognized by DPDA because either number of a's and b's can be equal or either number of b's and c's can be equal. So, it can only be implemented by NPDA.

Thus, it is CFL but not DCFL.

Hence DCFLs are not closed under Union.

Unit 4 - DCFLs are not closed under Concatenation

It is possible that the suffix of a former DCFL matches the prefix of a later DCFL. The DPDA would not know which part it is currently processing and would need non-determinism to correctly recognize the strings of the newly formed language. Hence, DCFLs are NOT closed under concatenation.

Consider the following DCFLs for which we can easily construct a DPDA:

$L_1 = \{a^m b^n \mid m < n\}$

$L_2 = \{wcw^R \mid w \in \{a, b\}^*$

Consider language L which represents the concatenation of these two languages :

$L=\{a^m b^n \mid m < n\} \cdot \{wcw^R \mid w \in \{a, b\}^*\}$

The string abbb·bbbacabbb is accepted, but how would you know on which b you should ”jump across” from one language to the other and stop consuming the b and start stacking it?

Therefore, $L=\{a^m b^n \mid m < n\} \cdot \{wcw^R \mid w \in \{a, b\}^*\}$ is NOT a DCFL but CFL.

Automata Formal Languages and Logic

Unit 4 - DCFLs are not closed under Star Closure

Let $L=L_1 \cup L_2 = \{ca^nb^n \mid n \in \mathbb{N} \cup \{0\}\} \cup \{a^mb^{2m} \mid m \in \mathbb{N}\}$, which is a deterministic context free language thanks to the letter c in front of every word in L_1. If L^* is the Kleene closure of L, any string $x \in L^*$ should be a concatenated permutation of words (let's say y_i) which belong to L , with the concatenation repeated numerably many times, zero included.

Now let $y_1=c$ and $y_2=ab^2$, which are both clearly in L. This can be seen by letting $n=0$ and $m=1$ in the sets L_1 and L_2. Because of this, the word cab^2 is in L^* , which can be obtained by taking the first iteration to generate c and second iteration to generate ab^2 from the set and forming a concatenation of these two words.

As both parts of the union that form L are DCFLs, they are both recognized by the deterministic pushdown automata (DPDA). In order to recognize the word $y_1y_2=cab^2$, you would have to form a new automaton, by connecting the individual automata appropriately. However, this connection cannot be made deterministically.

Automata Formal Languages and Logic

Unit 4 - DCFLs are not closed under Reversal

Example 1 :

Consider the language L = $WcW^R(a+b+c)*$, where $W \in (a+b)^+$; this is deterministic because you can write a deterministic PDA to accept simple palindromes of the form W^RcW and modify the accepting state to loop to itself on any of a, b and c.

The reverse of the language L, denoted by $L^R = (a+b+c)^*WcW^R$, where $W \in (a+b)^+$; this is non-deterministic because you don't know where the WcW bit starts.

Hence, DCFLs are not closed under reversal.

Example 2 :

$L = \{x0y \mid |x| = |y|, x \in 1^*, y \in \{0, 1\}^*\}$

The language L is deterministic context-free.

A deterministic PDA makes use of the stack to check to see if the two sides have the same length with the first occurrence of a 0 considered as the middle symbol of the input.

$L^R = \{y0x \mid |x| = |y|, x \in 1^*, y \in \{0,1\}^*\}$

Reversing the language will present difficulty for a deterministic PDA.

While a machine can easily identify the first occurrence of a 0, it cannot decide which occurrence of 0 is the last occurrence unless the machine has finished reading the whole input.

The class of DCFLs is closed under complementation.

The DPDA must be complete in order to take complement.

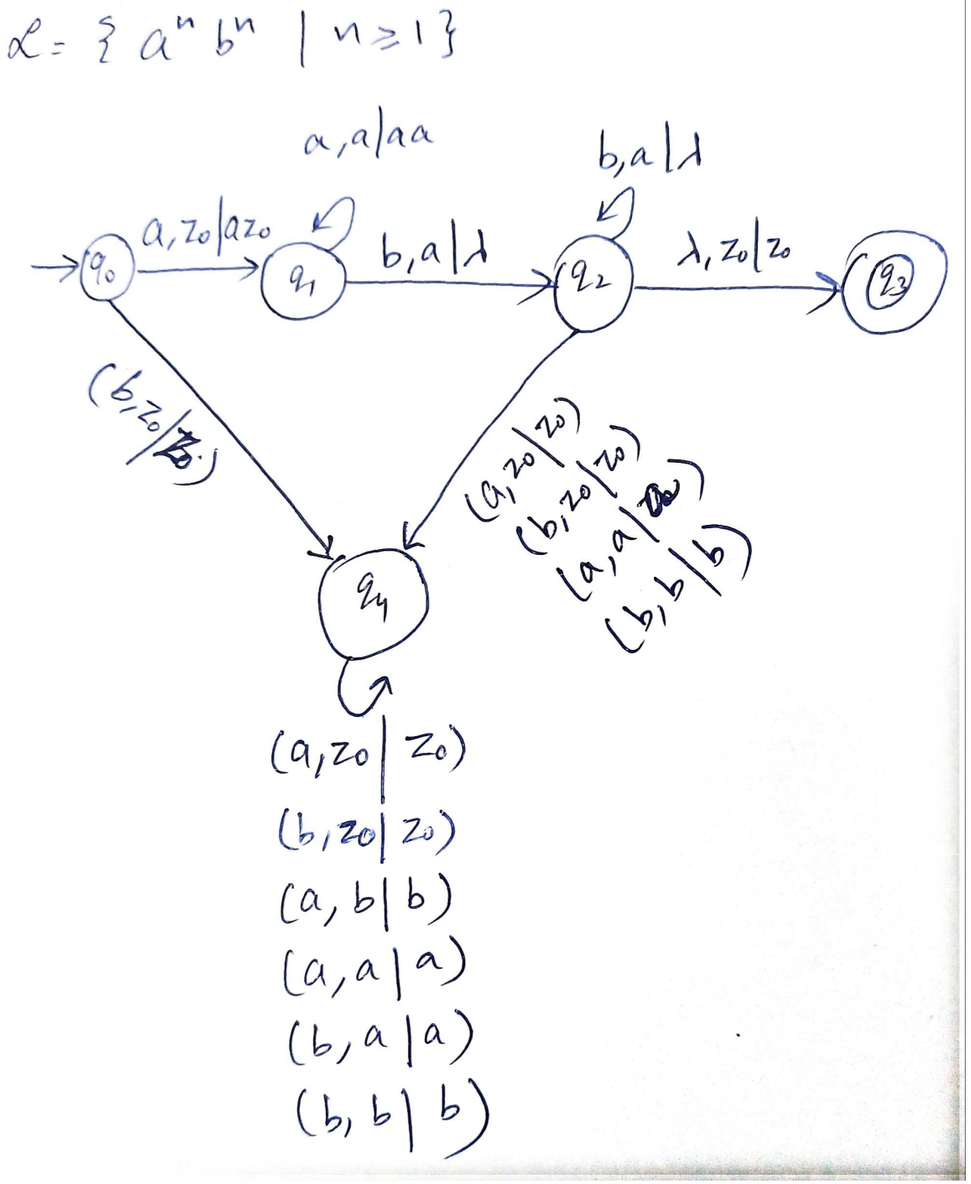
Introduce a dead state for invalid configurations and then toggle final and non-final states.

Automata Formal Languages and Logic

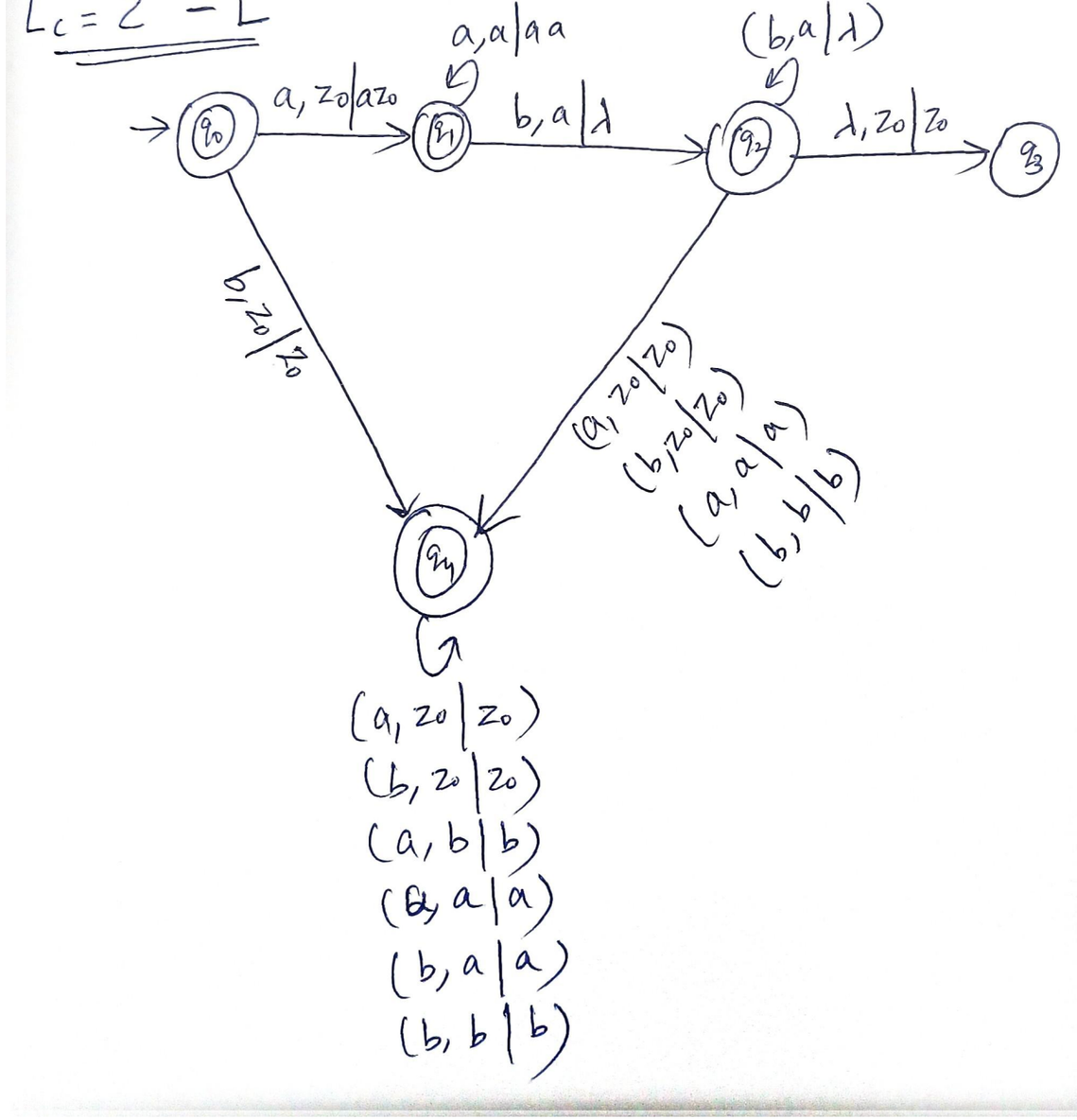
Unit 4 - DCFLs are closed under Complement

Example:

$L = \{ a^n b^n \mid n \geq 1 \}$



$L_c = \Sigma^* - L$



Automata Formal Languages and Logic

Unit 4 - Interesting question on Closure properties

What is the complement of the Context free language L which represents an even length palindrome given as :
$L = \{WW^R, W \{a,b\}^*\}$

Is the complement of L context free?

If L and its complement are context free, does that mean its a DCFL?

Automata Formal Languages and Logic

Unit 4 - Interesting questions on Closure properties

Solution:

WW^R means even length palindrome so its complement must accept

1) Odd length string

2) Even length string which are not palindrome

So here we can get help of non determinism, we perform push and at arbitrary point X assuming it to be half of string now we pop so there are few possibility

1) at least one symbol did not match (success)

2) Stack is empty and no input string is remaining to read (failure)

case 1 is definitely true for odd length string thus accepting but it also help in accepting even length string which are not palindrome.

case 2 is true when string is even length palindrome so thus at the end we will make a transition to non final state.

Automata Formal Languages and Logic

Unit 4 - Decidable Questions about CFLs

Is it possible to write an algorithm to answer the following about context free language(s)??

1. **Given a CFL L, is it possible to determine if L is infinite?**
   Sol :- if the dependency graph of Non-terminals have a cycle.

2. **Given a CFL L, is it possible to determine if L is empty i.e. $L = \Phi$?**
   Sol :- if start symbol of Grammar for L is useless.

3. **Given a CFL L, is it possible to determine if L generates everything, i.e. $L = \Sigma^*$?**
   Sol :- Decidable for DCFLs but not for CFLs as DCFLs are closed under complement.

4. **Given a CFG G and a string w, is it possible to determine if $w \in L(G)$?**
   Sol :- Using CYK

Automata Formal Languages and Logic

Unit 4 - Meaning of Undecidability

Undecidability means there is no algorithm that provides correct answer for all inputs.

The following properties of CFLs are undecidable:

1. **Given a CFG G, is it possible to determine if G is ambiguous?**
   Sol :- Not possible

2. **Given two CFLs L_1 and L_2, is it possible to determine if $L_1 = L_2$ i.e. if they generate the same language?**
   Sol :- If not, we will eventually get an answer. If yes, we will never get an answer as both the languages are infinite.

3. **Given two CFLs L1 and L2, is it possible to determine if $L_1 \cap L_2 = \Phi$?**
   Sol :- if the languages have string in common, we will eventually get an answer, if not we will wait forever as the languages are infinite.

THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724

PES UNIVERSITY ONLINE

Automata Formal Languages & Logic

Preet Kanwal

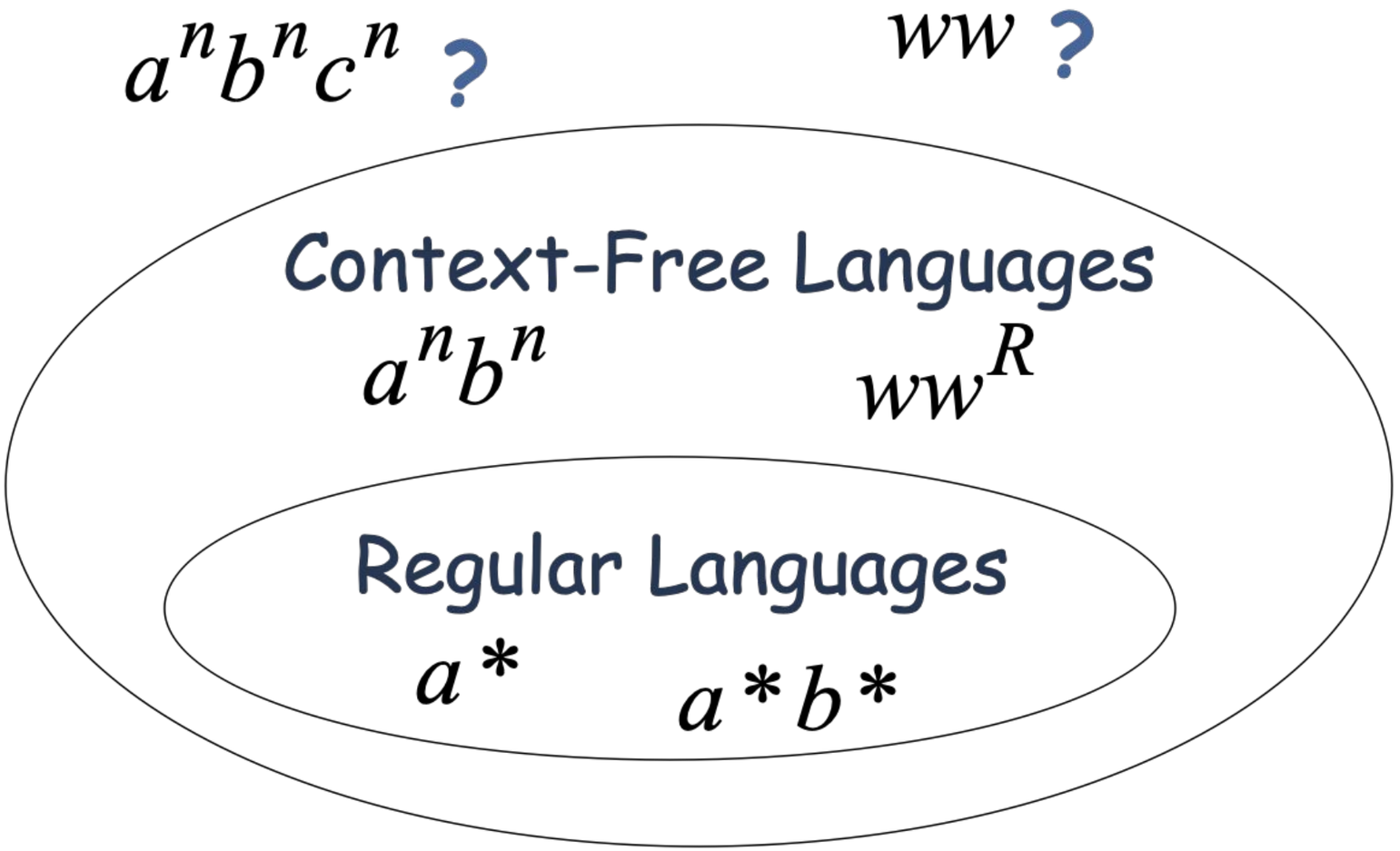
Department of Computer Science & Engineering

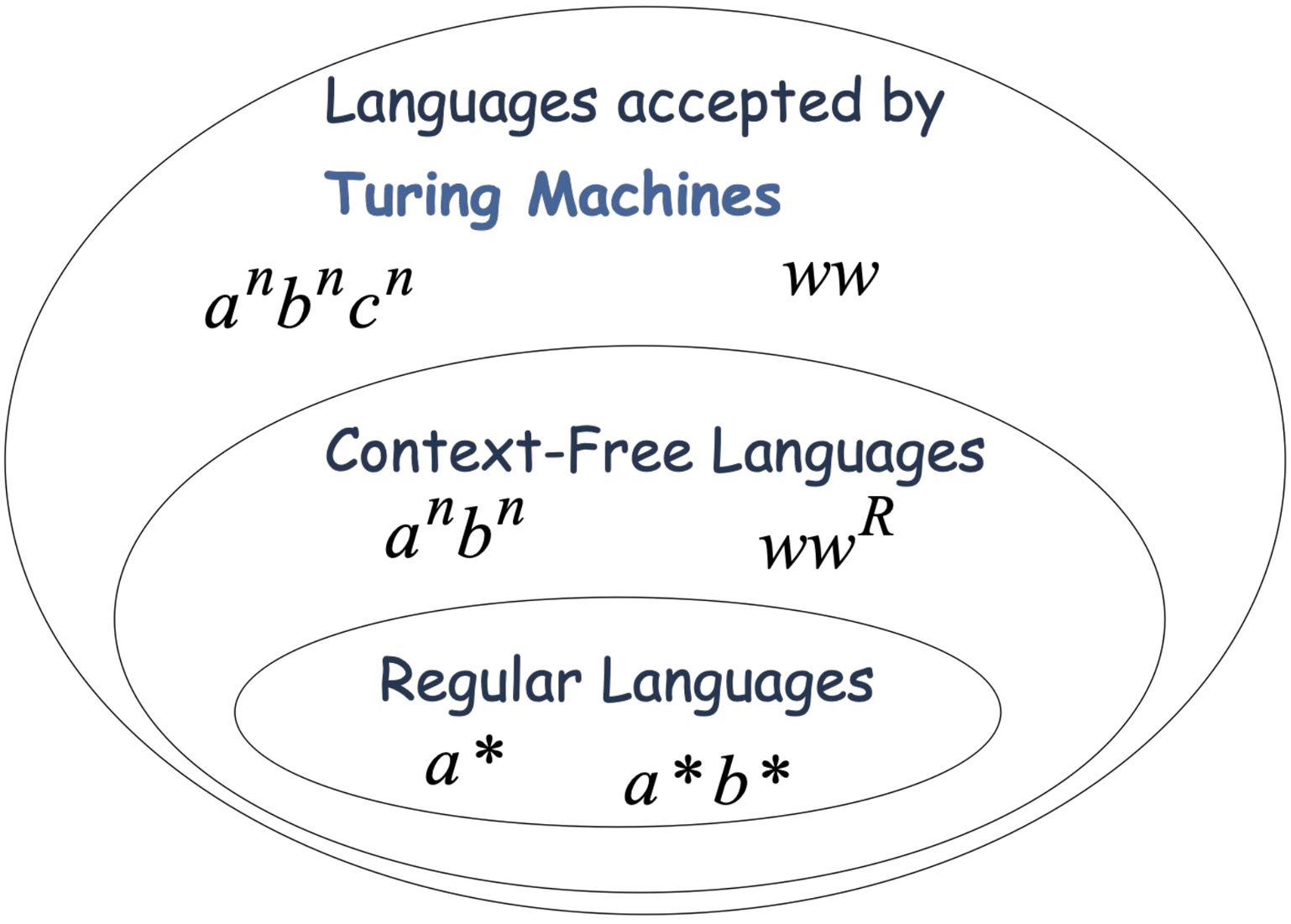
Automata Formal Languages & Logic

Unit 4 - Turing Machines

Preet Kanwal

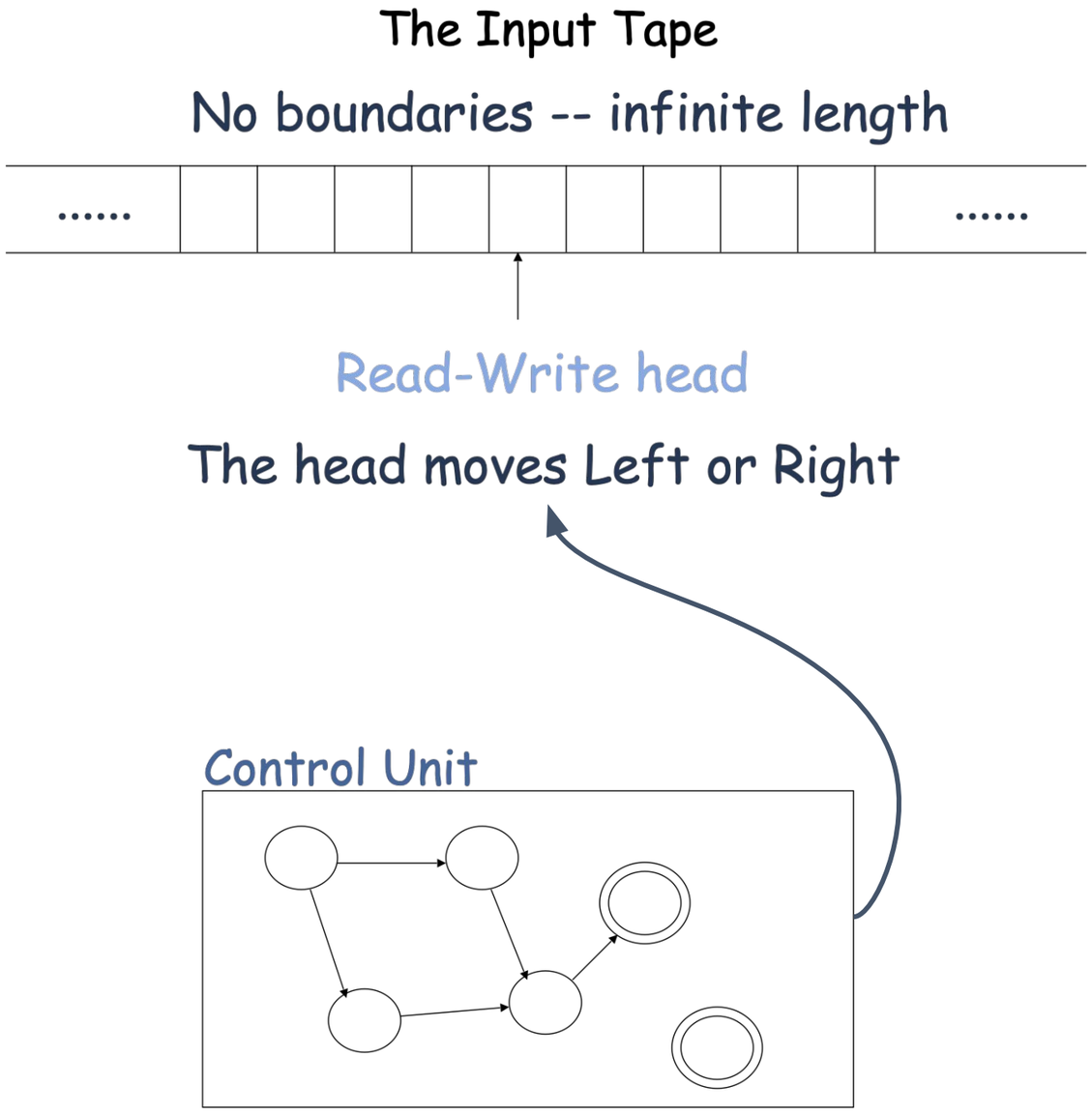
Department of Computer Science & Engineering


$a^n b^n c^n$?
ww ?
Context-Free Languages
$a^n b^n$
ww^R
Regular Languages
$a*$
$a*b*$


Languages accepted by
Turing Machines
$a^n b^n c^n$
ww
Context-Free Languages
$a^n b^n$
ww^R
Regular Languages
$a*$
$a*b*$

Automata Formal Languages and Logic

Unit 4 - A Standard Turing Machine

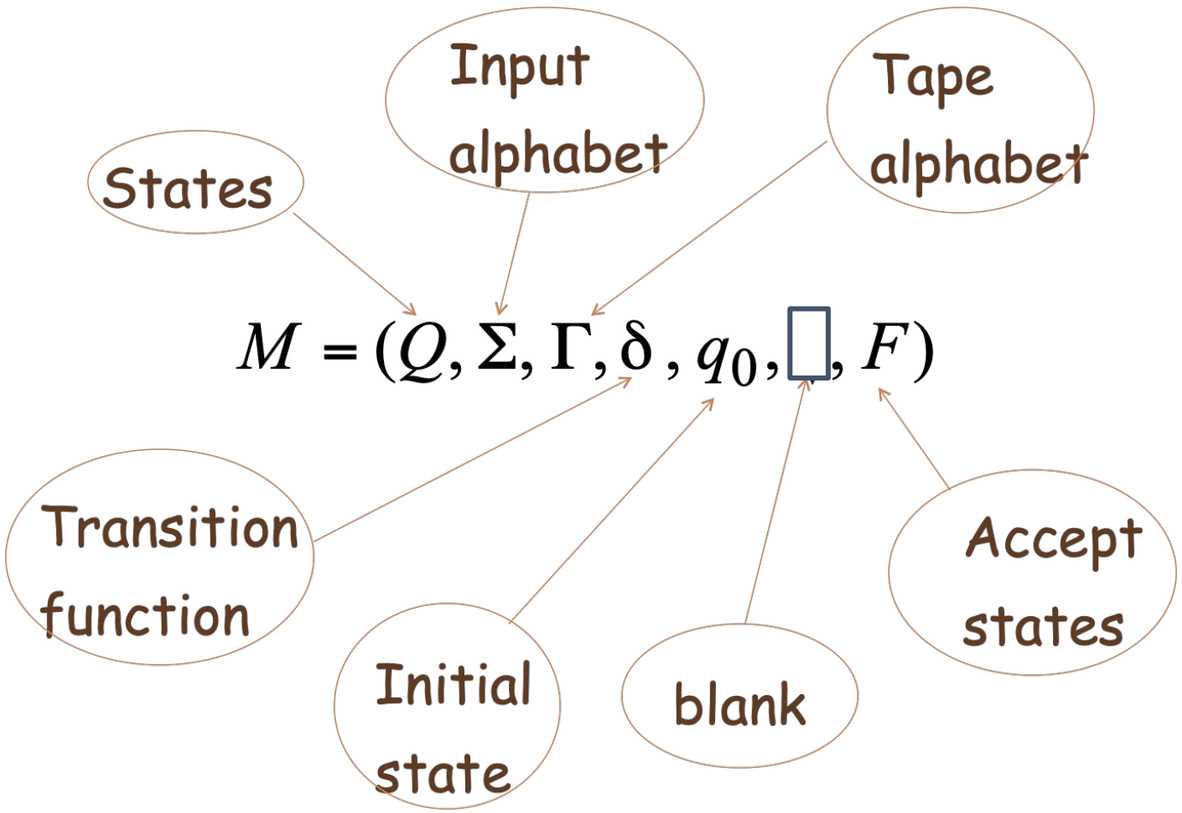


A Standard Turing Machine has the following components:

1. **Infinite Input tape - unbounded in both directions**
2. **Control Unit**
3. **R/W head - changes position in each move**

- **By convention, input string is preloaded on to the tape.**
- **Special markers can be used to indicate start and end of the input string.**

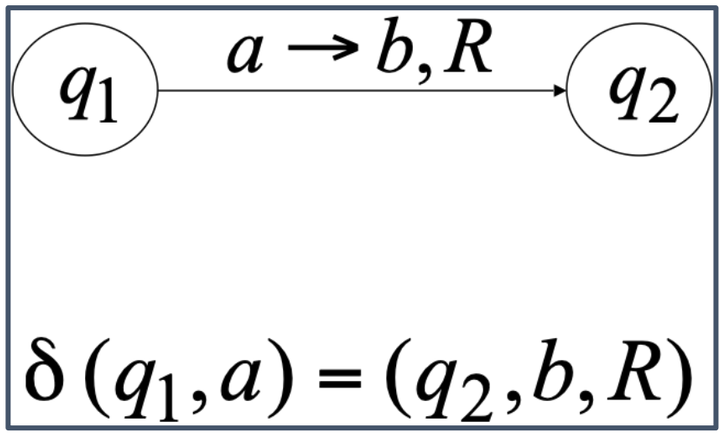
Turing Machine M, is a 7-tuple :

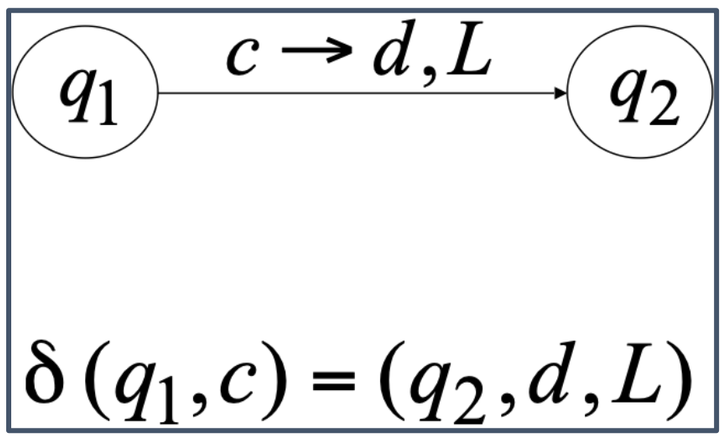


$\Sigma \subseteq \Gamma$

$\square \in \Gamma$

$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$



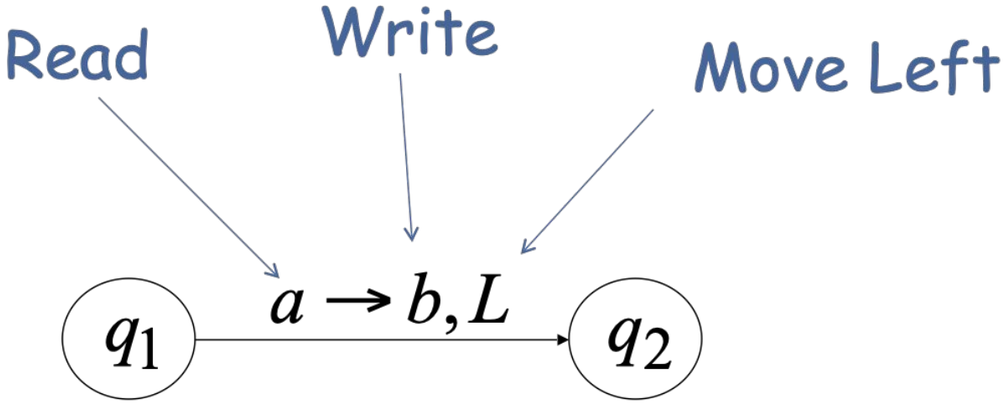


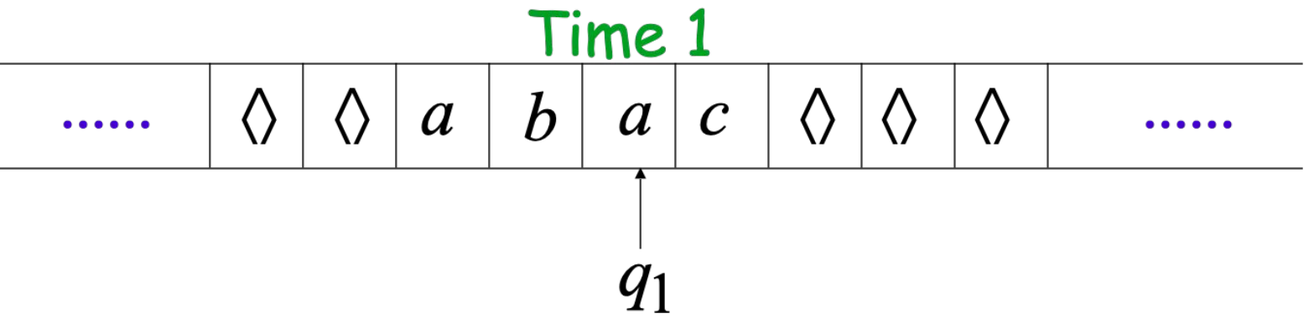
Note:
Textbook specifies the transition as:
a,a,R

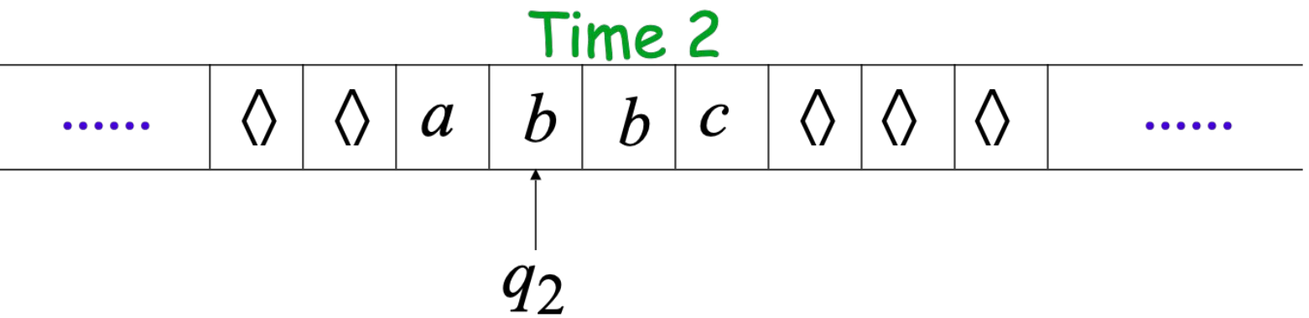
Other accepted notations:
a,a|R
or
a|a|R
or
a|a,R

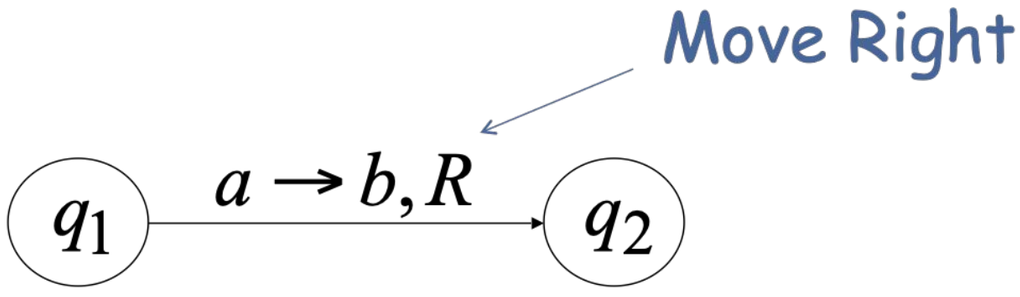
Automata Formal Languages and Logic

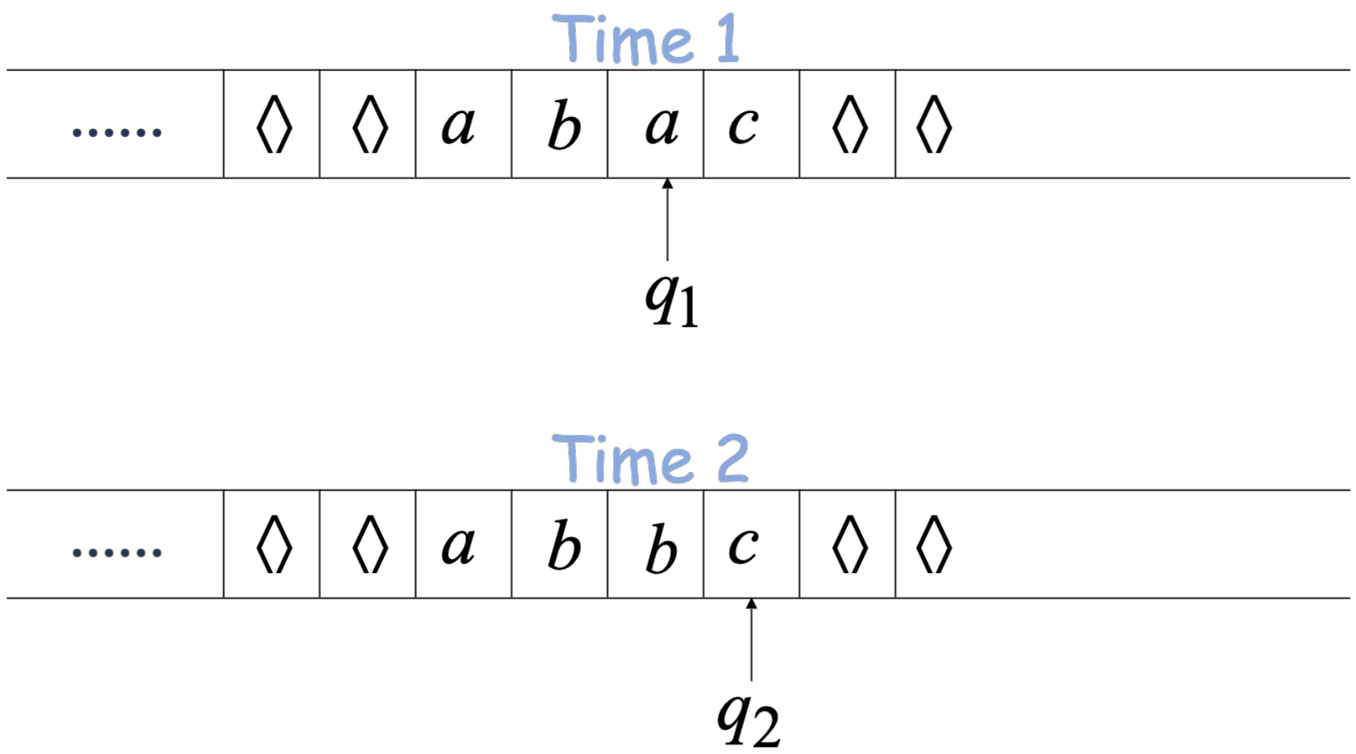
Unit 4 - Transition in a Turing Machine

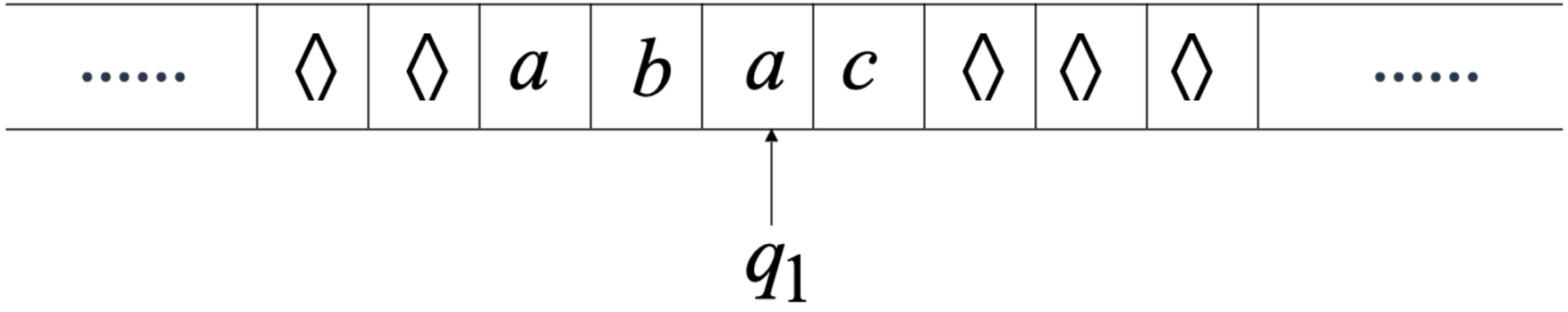


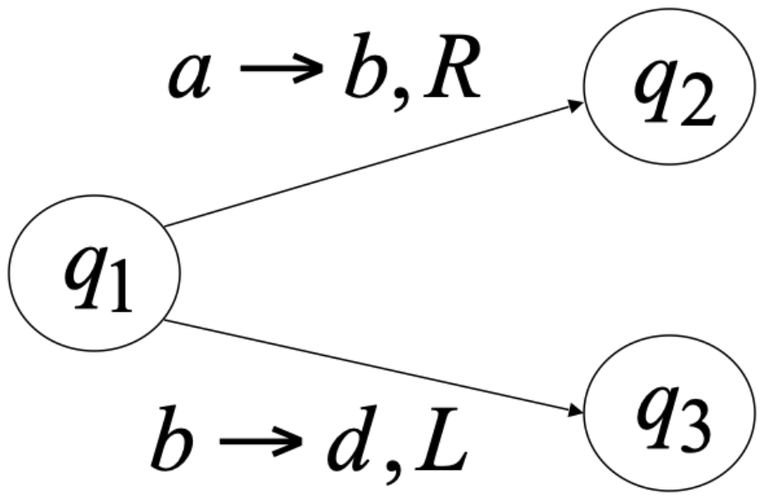





Move Right
q_1
$a \rightarrow b, R$
q_2


Time 1
.....
◊
◊
a
b
a
c
◊
◊
q_1
Time 2
.....
◊
◊
a
b
b
c
◊
◊
q_2





Allowed:

No transition

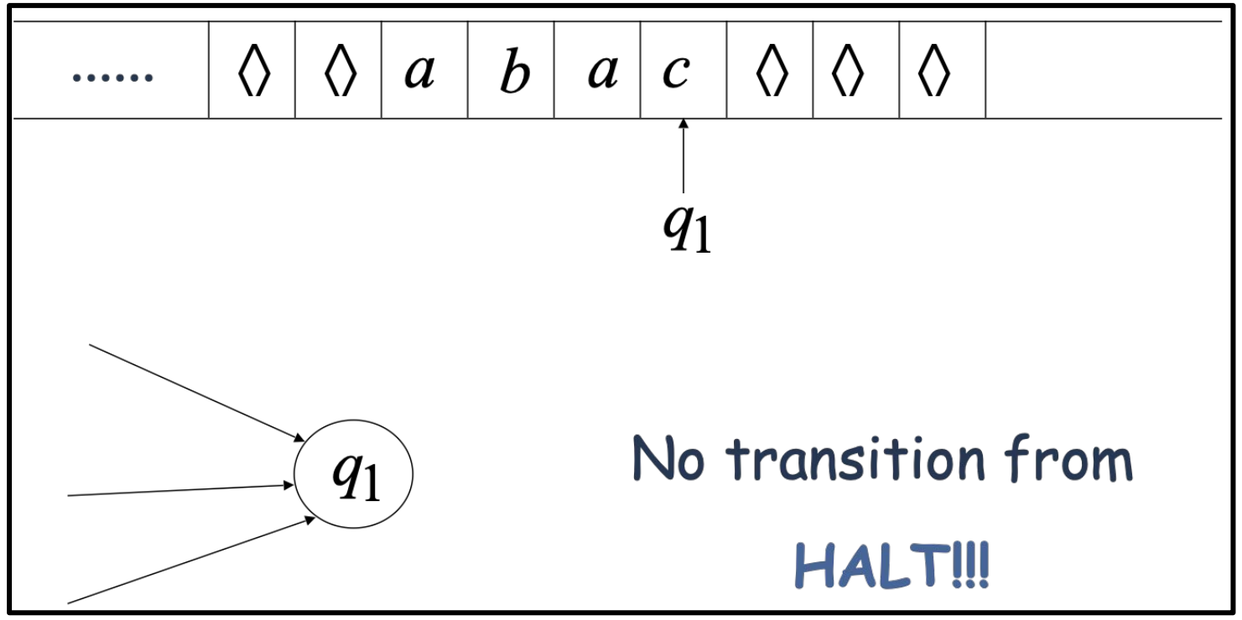
for input symbol c

Automata Formal Languages and Logic

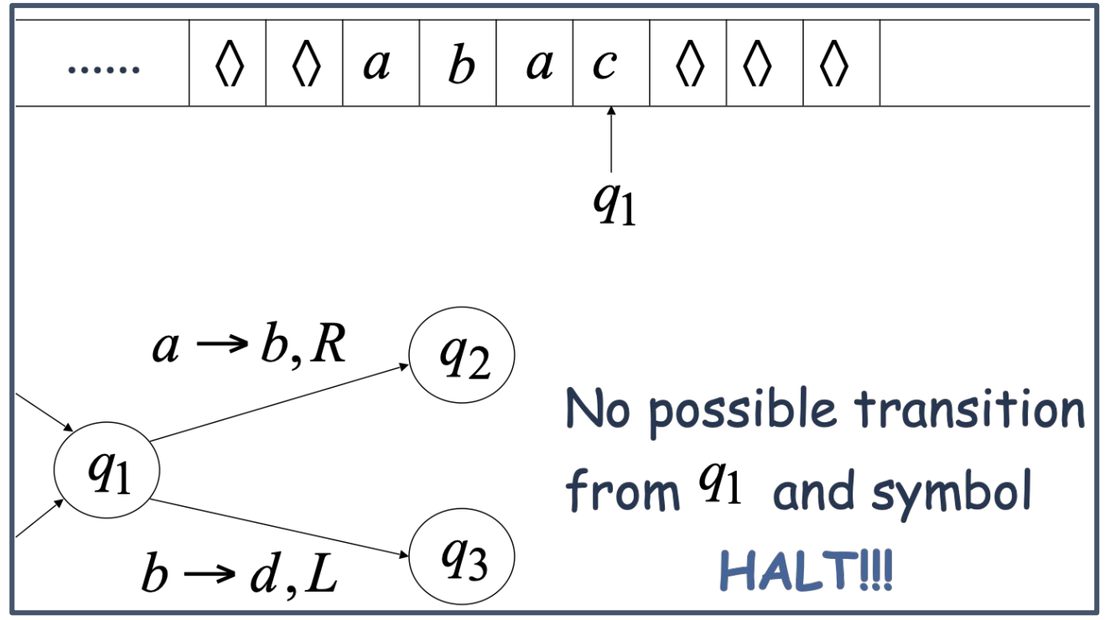
Unit 4 - Halting

The machine halts in a state if there is no transition to follow.

For example :



or



- Accepting states have no outgoing transitions.
- The machine halts and accepts.

Allowed

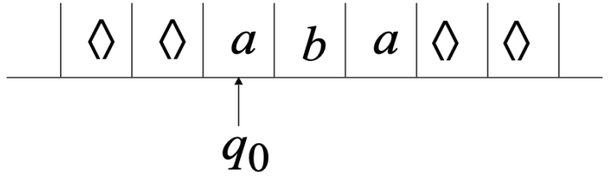
Not Allowed

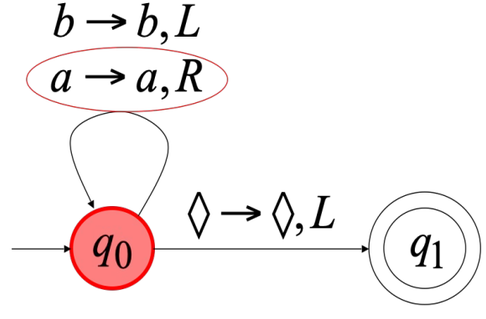
Accept Input string	→	If machine halts in an accept state
Reject Input string	→	If machine halts in a non-accept state or If machine enters an infinite loop

Automata Formal Languages and Logic

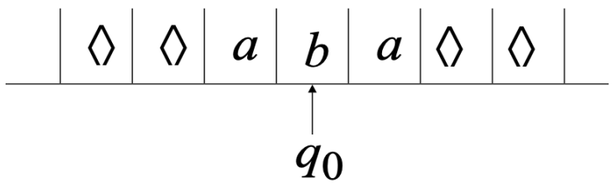
Unit 4 - Infinite Loop Example

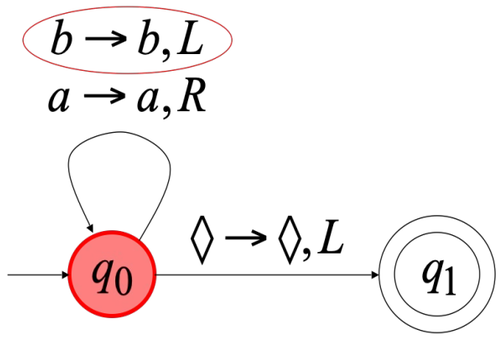
Time 0



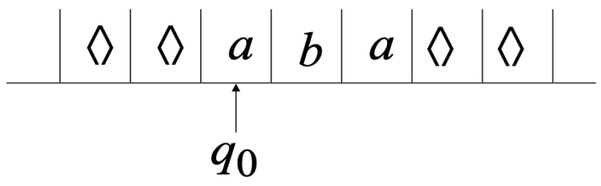


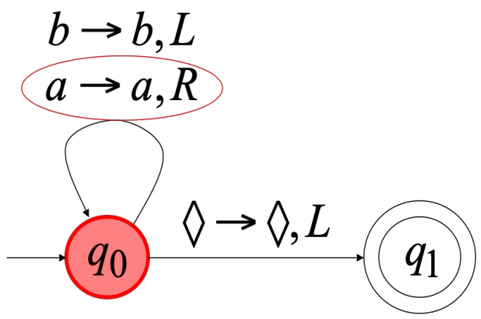
Time 1





Time 2

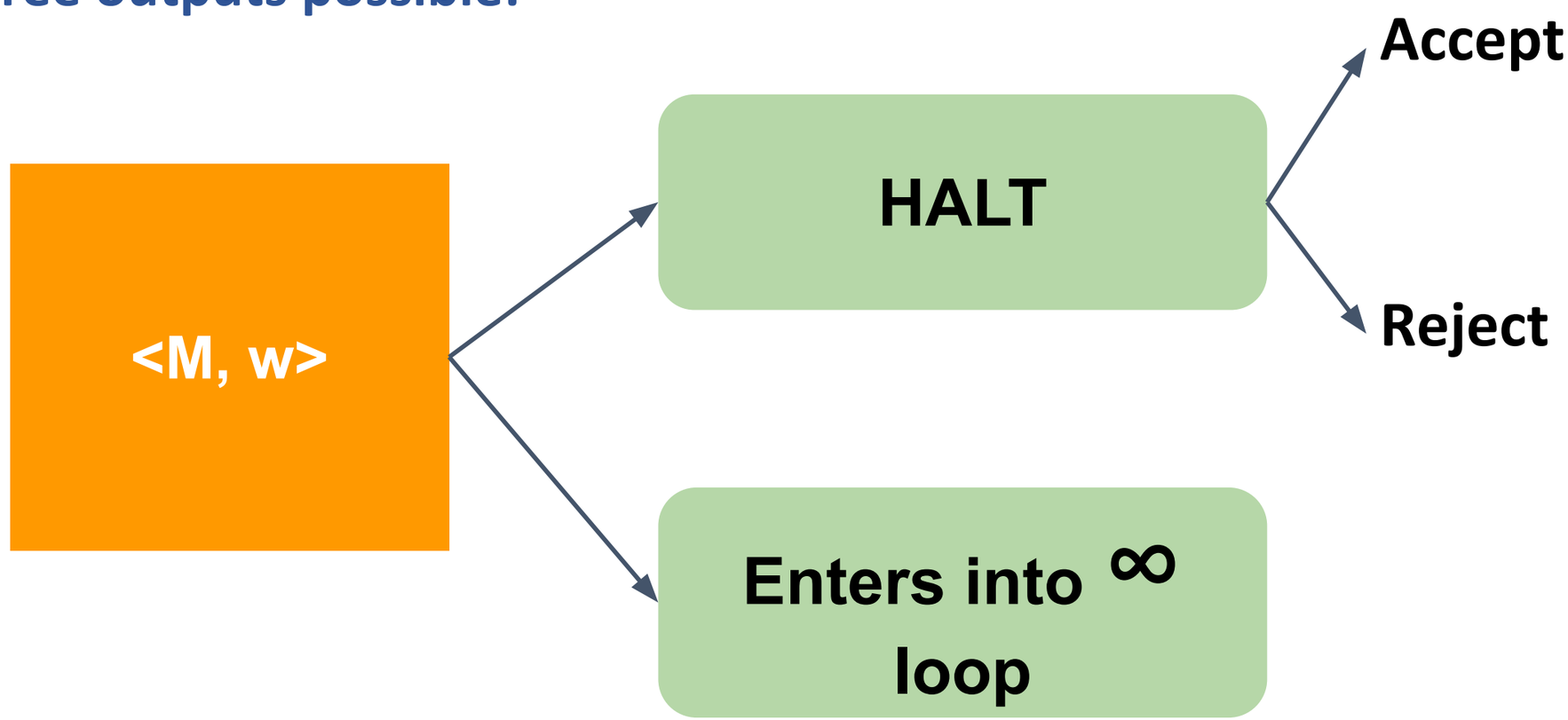




Because of the infinite loop:

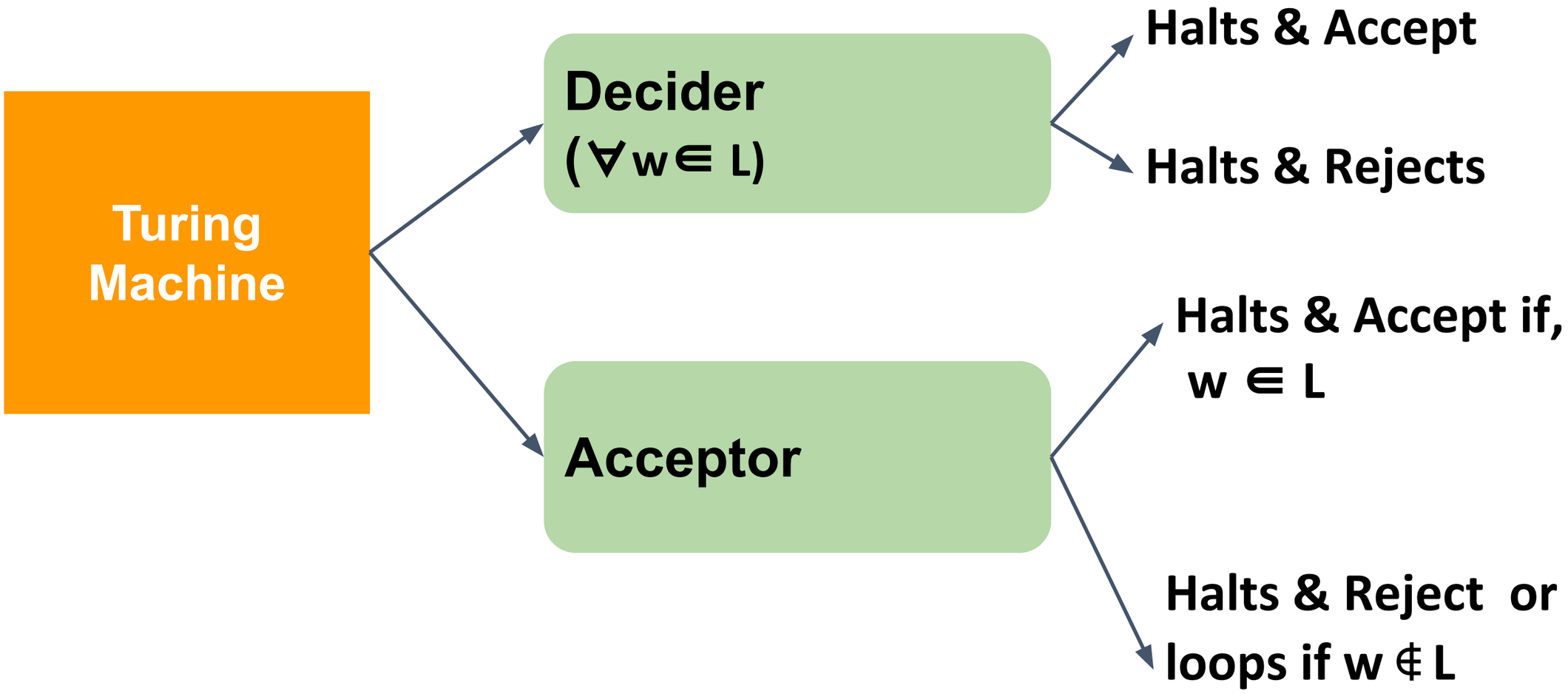
- **The accepting state cannot be reached**
- **The machine never halts**
- **The input string is rejected**

Given a Turing Machine M and a string w, when we run w on M, there are three outputs possible:



PES UNIVERSITY ONLINE

Given a Turing Machine M and a string w, when we run w on M, there are three outputs possible:



Language recognized by a Turing Machine is called :

Turing Recognizable Language

or

Turing Acceptable Language

or

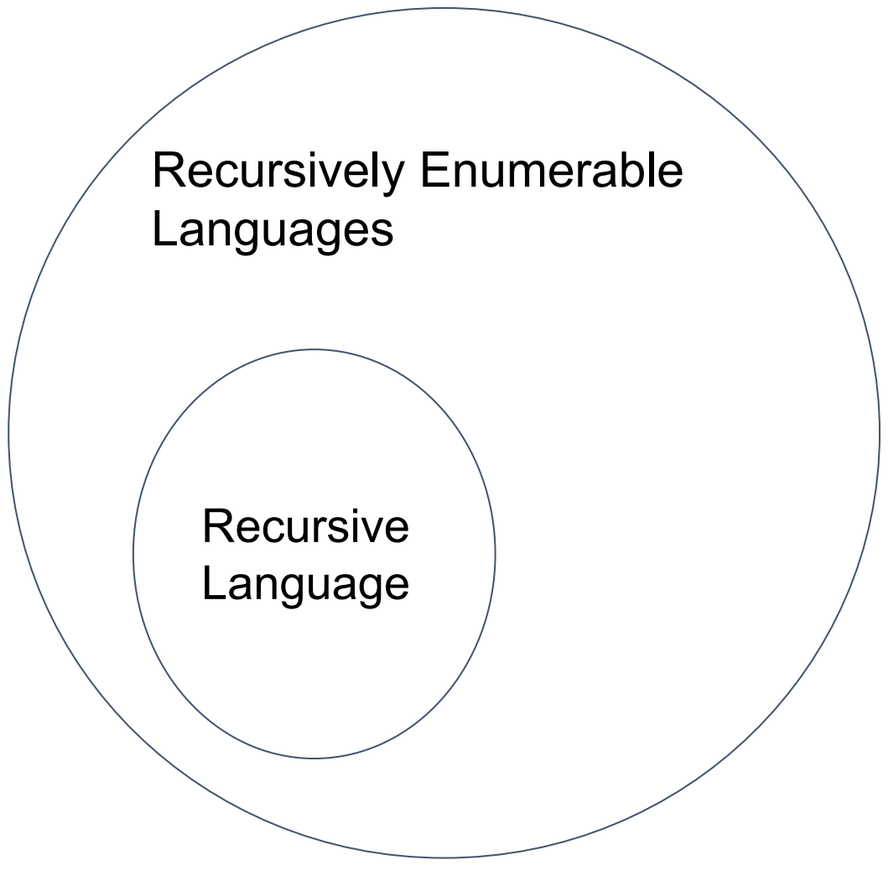
Recursively enumerable Language

Turing Machine Decider:

Turing Machines that halt on all inputs.

Language recognized by a Turing Machine Decider is called Recursive Language.

Recursive languages are closed under Complement.



Computation :

- It is a series of moves a Turing Machine makes given an input until it halts in some configuration.

Computable Function :

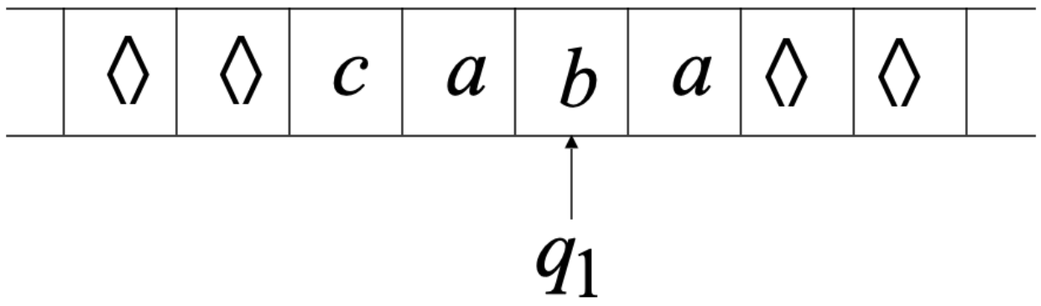
- A function f is computable if we can construct a Turing Machine Decider for that function.

For example: Following functions are computable:

1. f(x) = 2x
2. f(x,y) = x + y
3. $$f(x,y) = \begin{cases} 1 & \text{if } x > y \\ 0 & \text{if } x \leq y \end{cases}$$

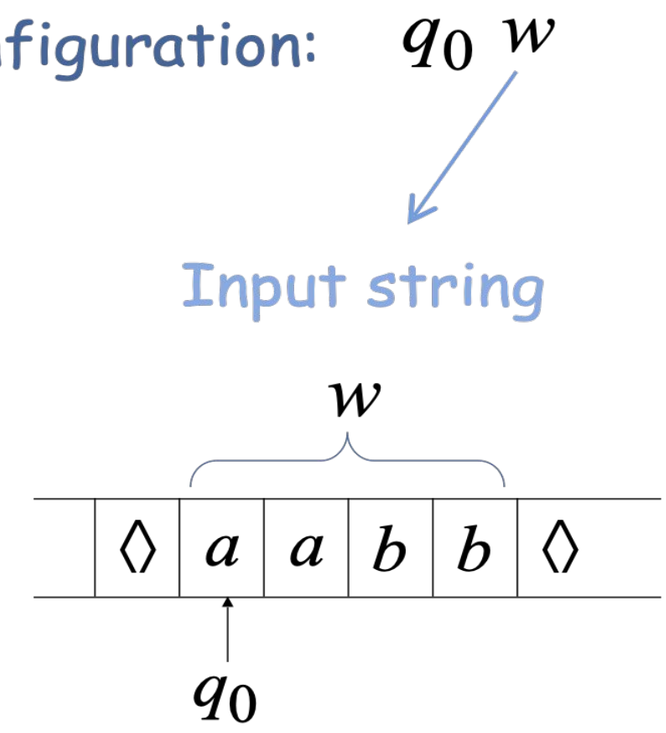
PES UNIVERSITY ONLINE

Current Configuration



Instantaneous description: $ca\ q_1\ ba$

Initial configuration: $q_0\ w$



Final Configuration : w q_f
where q_f is the final state

Automata Formal Languages and Logic

Unit 4 - Examples

Construct Turing Machine for the following examples:

1. a^nb^n, n>=1. Trace the input string : aabb
2. $0^n1^n2^n$, n>=1
3. Given input : $a^nb^mc^k$ such that k=n-m. Write the difference(in terms of no. of c's) on the tape. If the difference is negative append a -ve sign. For example if k=3 write ccc on the tape. If k=-2, write -cc on the tape. (Language of subtraction). Input and output must be separated by a ␣.
4. Given input : 0^m10^n, write 0^{m*n} as output on the tape (Language of Unary multiplication). Input and output must be separated by a ␣.
5. $0^n1^{n\hat{}2}$, n>=1
6. $0^{2\hat{}n}$, n>=0
7. ww, w ∈ $\{a,b\}^+$
8. $0^{n\hat{}2}$, n>=1

Automata Formal Languages and Logic

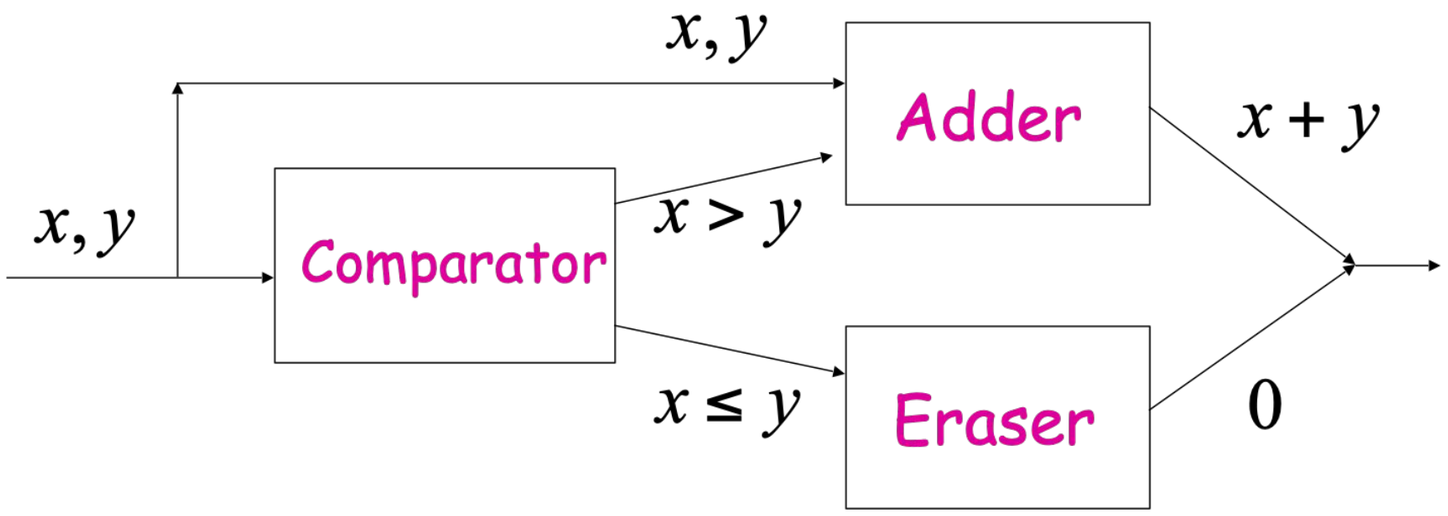
Unit 4 - Homework

Construct Turing Machine to recognize the following language:

1. $a^n b^m c^n d^m$, n>=1
2. Given input : $0^m 1 0^n$, write 0^{m+n} as output on the tape (Language of addition), m,n>=1
3. Given a string w ∈ $\{a,b\}^+$, write w as output on the tape separated by a blank. Final contents on the tape must look like : w␣w (Copy operation)
4. Given a string w∈ $\{a,b\}^+$, sort the symbols in the input string. For example if the input is babaa, output should be : aaabb. Final contents on the tape must look like: babaa␣aaabb
5. Given a string w∈ $\{a,b\}^+$, write w^R as the output on the tape (Reversing a string).Final contents on the tape must look like : w␣w^R

Example:

$$f(x,y) = \begin{cases} x+y & \text{if } x > y \\ 0 & \text{if } x \leq y \end{cases}$$



By combining Turing Machines that perform simple tasks, complex algorithms can be implemented.

The Church-Turing Thesis claims that every effective method of computation is either equivalent to or weaker than a Turing machine.

Alan turing came up with the notion that, what can be computed using Turing machine is known as computable.

All variations of the Turing Machine are equivalent in computing capability.

Algorithmically computable means computable by Turing Machine.

Automata Formal Languages and Logic

Unit 4 - Turing test

Warning :

Completely different term.

Used in Artificial Intelligence.

It is a test to determine whether a computer or program has same kind of intelligence as that of a human.

Alan turing designed both Turing Machine and Turing Test. They are completely different.

THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724